Unidad 3 – Parte A: Lenguajes Tipo 2

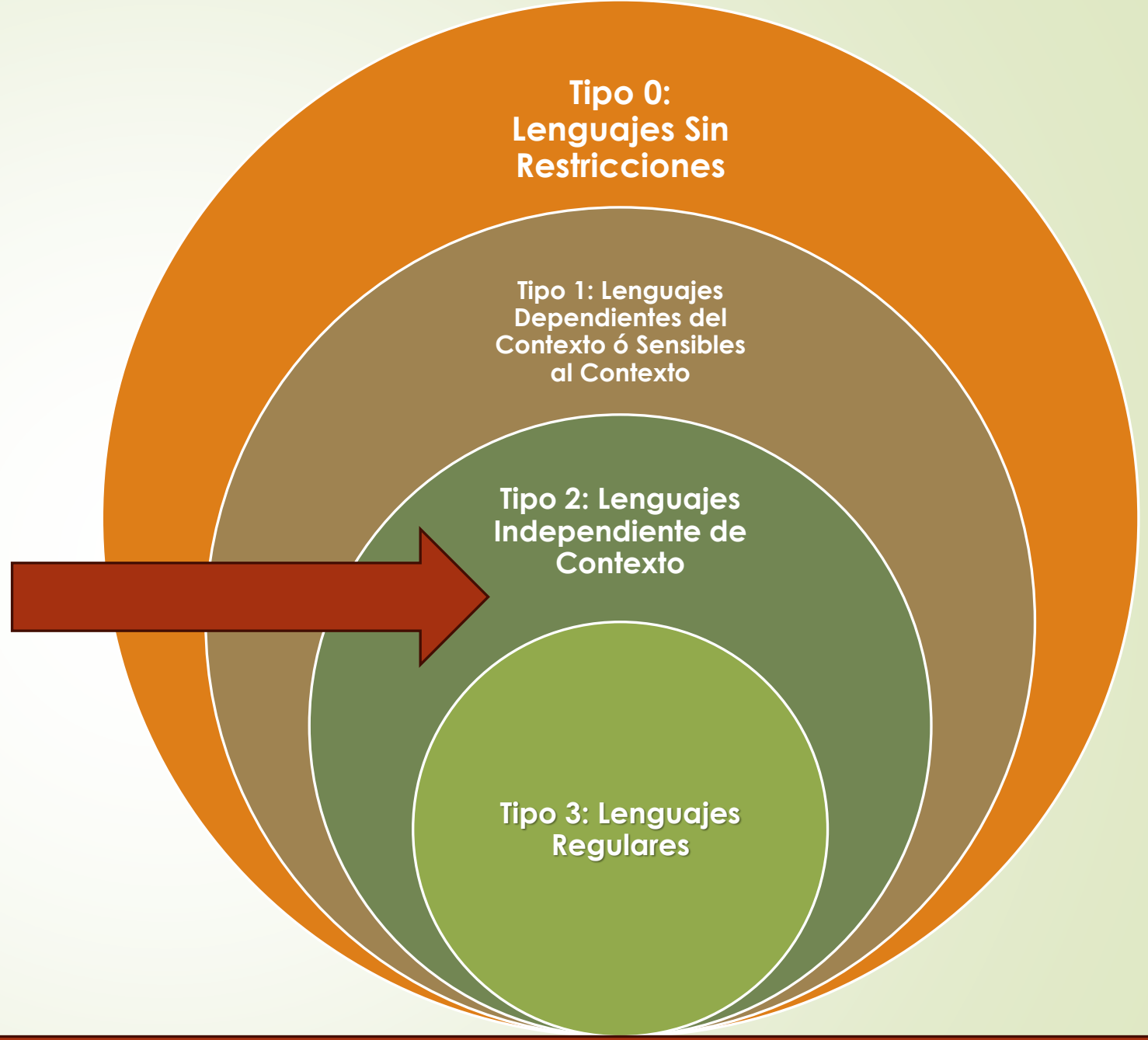
Gramáticas Independientes de Contexto

También llamadas: Gramáticas Libres de Contexto

Lenguaje Tipo 2

- Previamente: las Regex y los Lenguajes Regulares (Tipo 3): útiles para describir patrones lineales.
- *Más allá de las Regex*: aparecen lenguajes que requieren manejar estructuras jerárquicas.
  - Ejemplos:
    - Balanceo de paréntesis.
    - Correspondencia entre etiquetas de apertura y cierre en HTML/XML.
    - Frases con concordancia sujeto–verbo en lenguas naturales.
- Estos lenguajes se ubican en la Jerarquía de Chomsky como Tipo 2.
- Su poder expresivo es estrictamente mayor que el de los lenguajes regulares.

Jerarquía de Chomsky



Gramática

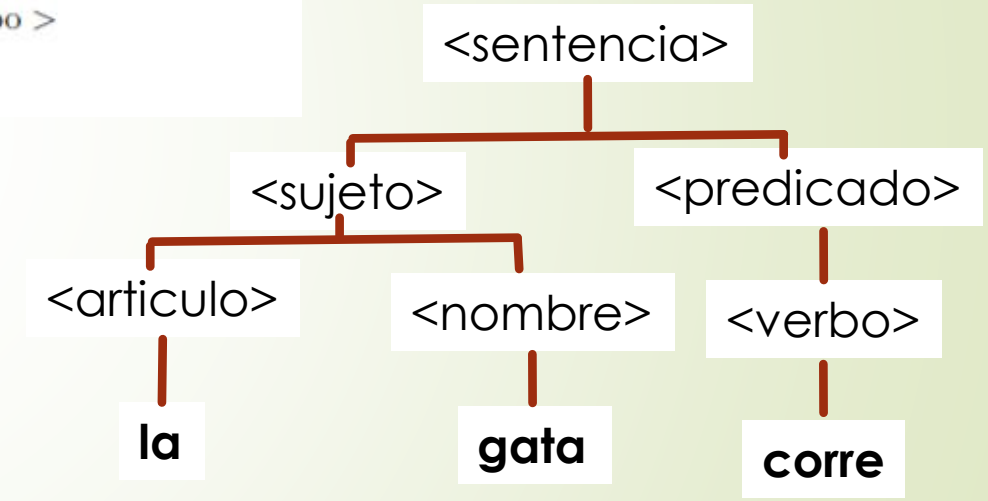
Ejemplo con un lenguaje natural (castellano) – Tomado del libro

"Estamos familiarizado con el concepto tradicional de gramática que, de forma intuitiva, podríamos considerar como un conjunto de reglas que nos indican qué es correcto y qué no lo es en un lenguaje natural. Con el fin de acercarnos a una definición más formal comenzaremos con un ejemplo en la lengua castellana.
La gramática debe describir la estructura de las frases y de las palabras de un lenguaje. Veamos una serie de reglas muy sencillas para comprobar la frase". **La gata corre**

Reglas gramaticales:

1. < sentencia >::=< sujeto >< predicado >
2. < sujeto >::=< articulo >< nombre >
3. < predicado >::=< verbo >< complemento >
4. < predicado >::=< verbo >
5. < articulo > ::= *el*
6. < articulo > ::= *la*
7. < nombre > ::= *perro*
8. < nombre > ::= *gata*
9. < verbo > ::= *corre*
10. < verbo > ::= *come*
11. < complemento > ::= *deprisa*
12. < complemento > ::= *mucho*

1 < **sujeto** > < predicado >
2 < articulo >< nombre > < **predicado** >
4 < **articulo** > < nombre >< verbo >
6 la< **nombre** > < verbo >
8 la gata< **verbo** >
9 la gata corre



Jurado Málaga, J. Teoría de autómatas y lenguajes formales. Universidad de Extremadura, Servicio de Publicaciones, 2008. ISBN 978-84-691-6345-0
https://dehesa.unex.es/entities/publication/51da7af5-d5fa-4472-b018-e93e4c375d54

Gramática Independiente de Contexto (GIC)
Definición

Los lenguajes libres de contexto serán definidos por medio de gramáticas libres de contexto, las cuales están formadas por 4 elementos:

- **Elementos Terminales (E_T):** Son los símbolos del alfabeto del lenguaje (por ejemplo: las letras que aparecen en las cadenas finales)
- **Elementos No Terminales (E_N):** Son símbolos que permiten realizan derivaciones y dar estructura a la gramática
- **Símbolo inicial (S):** También llamado "Start Symbol" es elemento inicial de la gramática
- **Conjunto de Producciones (P):** Son las reglas que conforman que conforman la gramática del lenguaje.

$E_N \cap E_T = \emptyset$

$S \notin E_N$

(E_N, E_T, S, P)

Elementos que pertenecen al lenguaje

Ejercicio de GIC

Dada las reglas de la gramática:

$S \rightarrow aS \mid Sb \mid \lambda$

1) Indicar: $\mathbf{(E_N, E_T, S, P)}$

$$G=(\{\}, \{a,b\}, S, P)$$

2) Definir el lenguaje

$$L=\{a^n b^m \quad n,m \geq 0\}$$

Ejercicio Propuesto 1 (tomado del libro)

Cadenas que se pueden derivar a partir de una gramática

$$S \rightarrow aS \mid bS \mid c$$

Respuesta: F V F V F

De acuerdo con la GIC anterior responda V o F

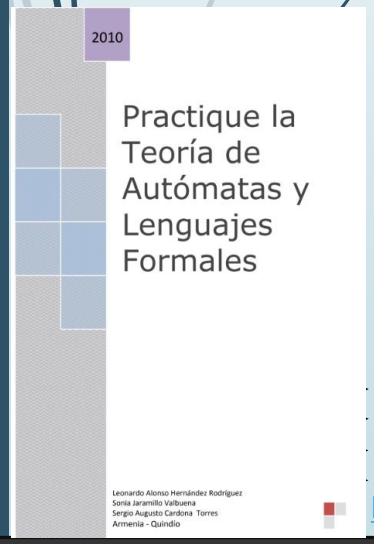
a. La GIC deriva la palabra *ca* _____

b. La GIC deriva la palabra *c* _____

c. La GIC deriva la palabra *cb* _____

d. La GIC deriva la palabra *abababbac* _____

e. La GIC deriva la palabra *a* _____



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez
Sonia Jaramillo Valbuena
Sergio Augusto Cardona Torres
Armenia - Quindío

https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

Ejercicio Propuesto 2 (tomado del libro)

Cadenas que se pueden derivar a partir de una gramática

$$S \rightarrow aSb \mid c$$

De acuerdo con la GIC anterior responda V o F

a. La GIC deriva la palabra *ac* _____

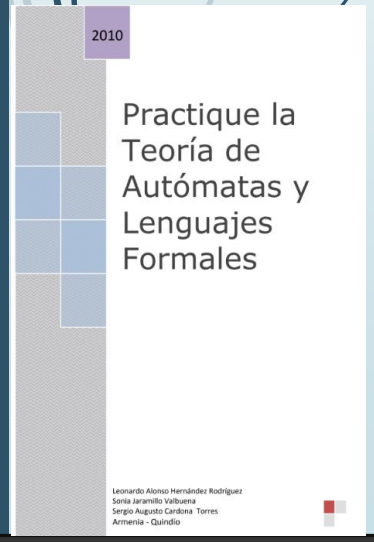
b. La GIC deriva la palabra *acbb* _____

c. La GIC deriva la palabra *abc* _____

d. La GIC deriva la palabra *aaaacbbbb* _____

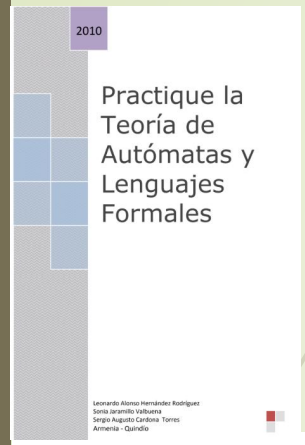
e. La GIC deriva la palabra *c* _____

Respuesta: F F F V V



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez
Sonia Jaramillo Valbuena
Sergio Augusto Cardona Torres
Armenia - Quindío
https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

Gramática Independiente de Contexto (GIC)- Derivación



La derivación es un proceso mediante el cual se deriva una palabra en varios pasos comenzando por el símbolo inicial de la GIC.
La siguiente es la derivación de la palabra bba utilizando la gramática del ejemplo.

$S \Rightarrow \mathbf{A}B \Rightarrow \mathbf{Ab}B \Rightarrow AbbB \Rightarrow bbB \Rightarrow bba$

El símbolo $\Rightarrow$ se lee: produce, deriva, genera.

Partimos de S y mediante cinco pasos llegamos a *bba*. Solo podemos realizar los pasos permitidos por la gramática. Por ejemplo en el segundo paso se reemplazó A por Ab, porque esto lo permite la producción A -> Ab.

$S \rightarrow AB$
$A \rightarrow aA$
$A \rightarrow Ab$
$A \rightarrow \lambda$
$B \rightarrow aB$
$B \rightarrow a$

Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez; Sonia Jaramillo Valbuena ; Sergio Augusto Cardona Torres ; Armenia - Quindío
https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

Pregunta agregada:

¿Qué reglas se utilizaron?

$$S \overset{1}{\Rightarrow} AB \overset{3}{\Rightarrow} AbB \overset{3}{\Rightarrow} AbbB \overset{4}{\Rightarrow} bbB \overset{6}{\Rightarrow} bba$$

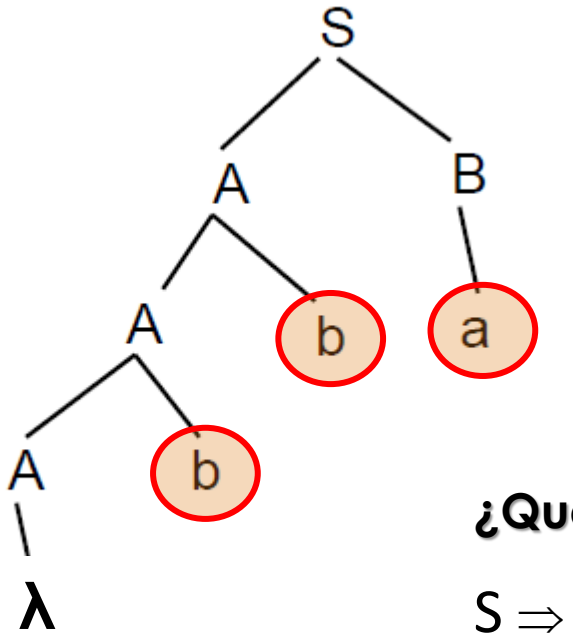
1 $S \rightarrow AB$
2 $A \rightarrow aA$
3 $A \rightarrow Ab$
4 $A \rightarrow \lambda$
5 $B \rightarrow aB$
6 $B \rightarrow a$

Se aplicaron las reglas: 1,3,3,4,6

Gramática Independiente de Contexto (GIC)- Derivación

La derivación de una palabra mediante una GIC también puede ser representada por un árbol denominado árbol de derivación o de análisis. Por ejemplo, la derivación anterior puede ser representada por el siguiente árbol de derivación:

El árbol siempre tendrá en su nodo raíz el símbolo inicial de la gramática (comúnmente lo llamamos S), los nodos hojas donde ya no se desprenden otros nodos son Elementos Terminales (ET) de la gramática. Los nodos intermedios corresponden a elementos no terminales (EN)



La cadena se obtiene leyendo los nodos hojas de izquierda a derecha: bba"

¿Qué reglas se utilizaron?

$$S \overset{1}{\Rightarrow} AB \overset{3}{\Rightarrow} AbB \overset{3}{\Rightarrow} AbbB \overset{4}{\Rightarrow} bbB \overset{6}{\Rightarrow} bba$$

Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez; Sonia Jaramillo Valbuena ; Sergio Augusto Cardona Torres ; Armenia - Quindío
https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

Gramática Independiente de Contexto (GIC)- Derivación

Formas de Derivación

Right-most es aquella derivación que en cada forma sentencial selecciona el no terminal de más a la derecha para derivar directamente.

Left-most es aquella derivación que en cada forma sentencial selecciona el no terminal de más a la izquierda para derivar directamente.

Gramática Independiente de Contexto (GIC)
Derivación

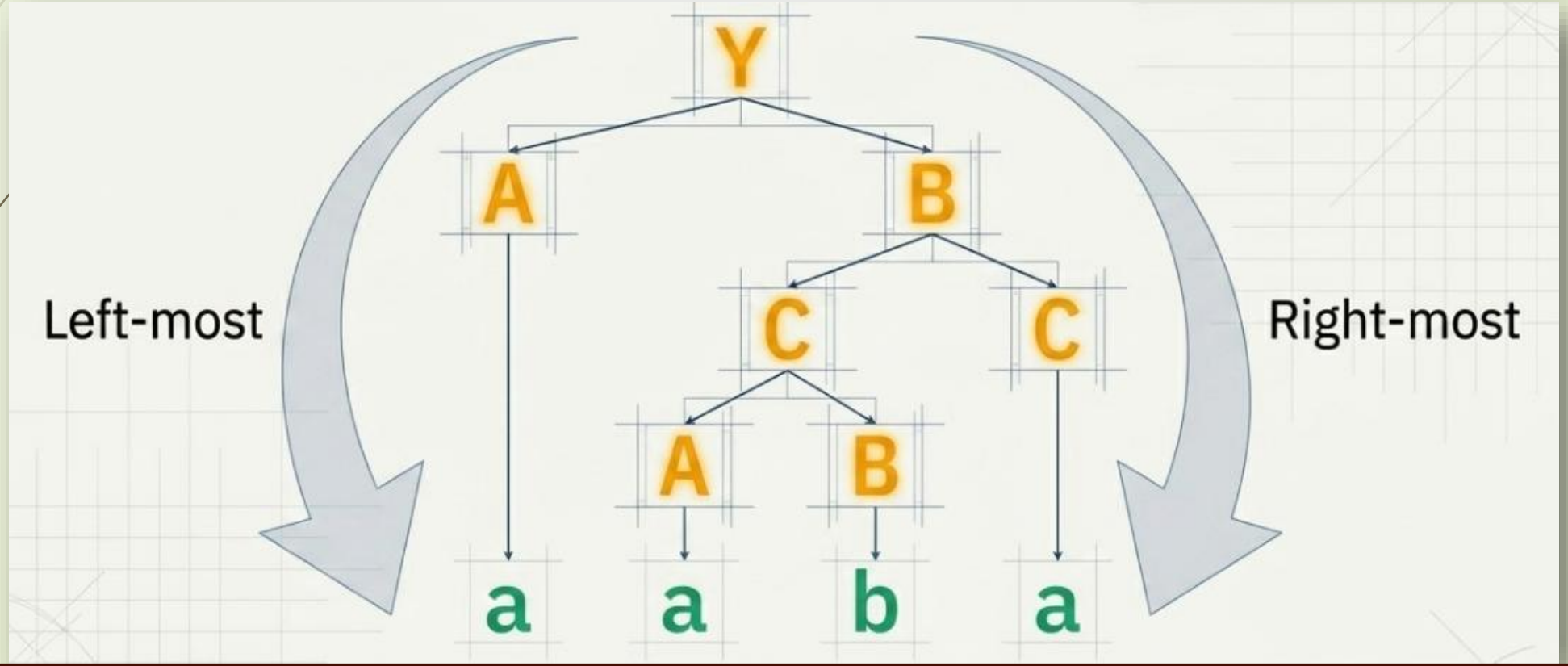
Reglas

1	Y → AB
2	Y → BC
3	A → BA
4	A → a
5	B → CC
6	B → b
7	C → AB
8	C → a

$$\boldsymbol{Y} \overset{1}{\rightarrow} \boldsymbol{A}B \overset{4}{\rightarrow} a\boldsymbol{B} \overset{5}{\rightarrow} a\boldsymbol{C}C \overset{7}{\rightarrow} a\boldsymbol{A}BC \overset{4}{\rightarrow} aa\boldsymbol{B}C \overset{6}{\rightarrow} aab\boldsymbol{C} \overset{8}{\rightarrow} \boldsymbol{aaba} \; (derivación\ left\ most)$$

$$\boldsymbol{Y} \overset{1}{\rightarrow} A\boldsymbol{B} \overset{5}{\rightarrow} AC\boldsymbol{C} \overset{8}{\rightarrow} A\boldsymbol{C}a \overset{7}{\rightarrow} AA\boldsymbol{B}a \overset{6}{\rightarrow} A\boldsymbol{A}ba \overset{4}{\rightarrow} \boldsymbol{A}aba \overset{4}{\rightarrow} \boldsymbol{aaba} \; (derivación\ right\ most)$$

7 pasos la derivación



Gramática Independiente de Contexto (GIC)
Recursividad

- ¿Cómo escribimos reglas para procesar un número de tamaño infinito sin tener que crear infinitas reglas?



Recursiva a Izquierda

```
<numero> ::= <numero> <digito>
```

Recursiva a Derecha

```
<numero> ::= <digito> <numero>
```

```
<suma> ::= <numero> + <suma>
```

GIC: Reglas Recursivas

Sintaxis para una cadena de sumas de números. Ejemplo : 12+3+248

<suma> ::= < numero >

<suma> ::= < numero > + <suma>

<numero> ::= <digito>

regla recursiva a derecha

<numero> ::= <numero> <digito>

regla recursiva a izquierda

<digito> ::= 0
<digito> ::= 1
<digito> ::= 2
<digito> ::= 3
<digito> ::= 4
<digito> ::= 5
<digito> ::= 6
<digito> ::= 7
<digito> ::= 8
<digito> ::= 9

(E_N, E_T, S, R)

E_N= {<numero>,<digito>}

E_T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +}

S= <suma>

R

Las **GIC** pueden mezclar recursividad por derecha y por izquierda en distintas producciones

GIC: Notación Compacta – Operador ó

Antes (13 reglas)

```
<suma> ::= <numero>
<suma> ::= <numero> + <suma>
<suma> ::= <numero> - <suma>

<digito> ::= 0
<digito> ::= 1
<digito> ::= 2
<digito> ::= 3
<digito> ::= 4
<digito> ::= 5
<digito> ::= 6
<digito> ::= 7
<digito> ::= 8
<digito> ::= 9
```



Después (13 reglas – 3 renglones)

```
<suma> ::= <numero>
   | <numero> + <suma>
   | <numero> - <suma>
<numero> ::= <digito>
   | <numero> <digito>

<digito> ::= 0 | 1 | 2 |
3 | 4 | 5 | 6 | 7 | 8 | 9
```

El símbolo de pipe (|) actúa como un conector lógico "O". Agrupa producciones con el mismo lado izquierdo.

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

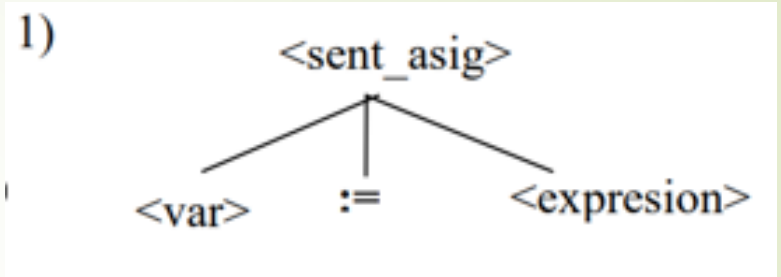
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

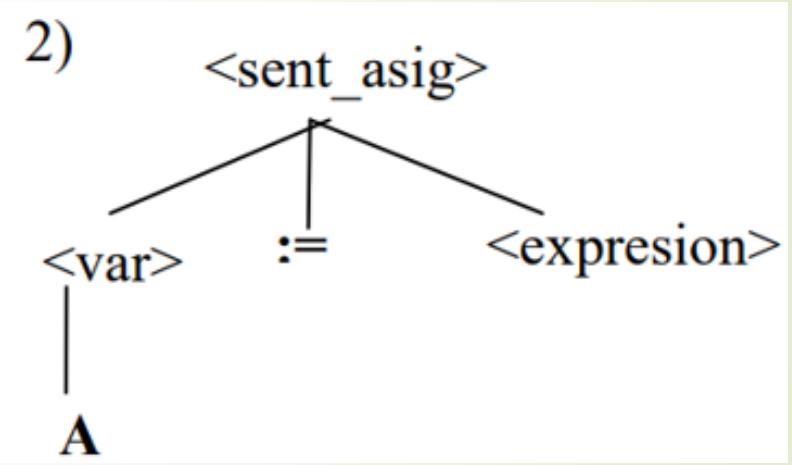
<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

1)



2)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

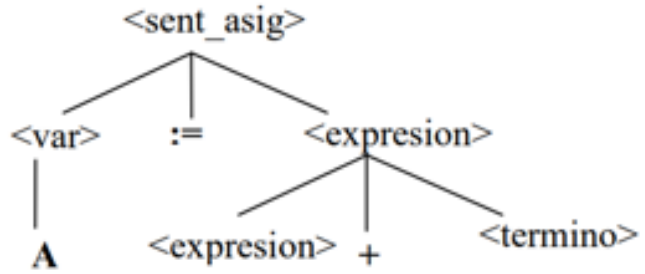
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

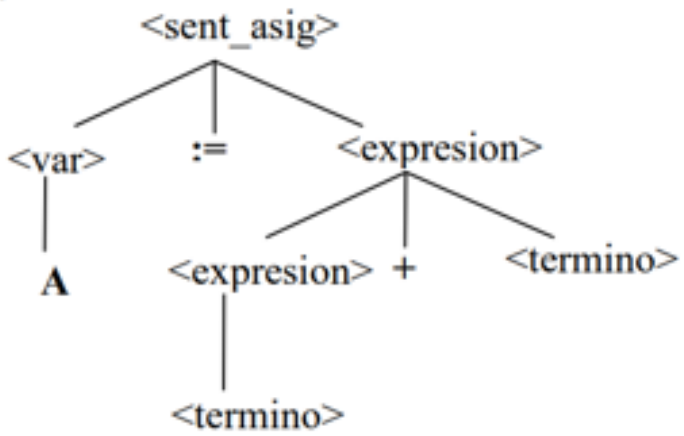
<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

3)



4)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

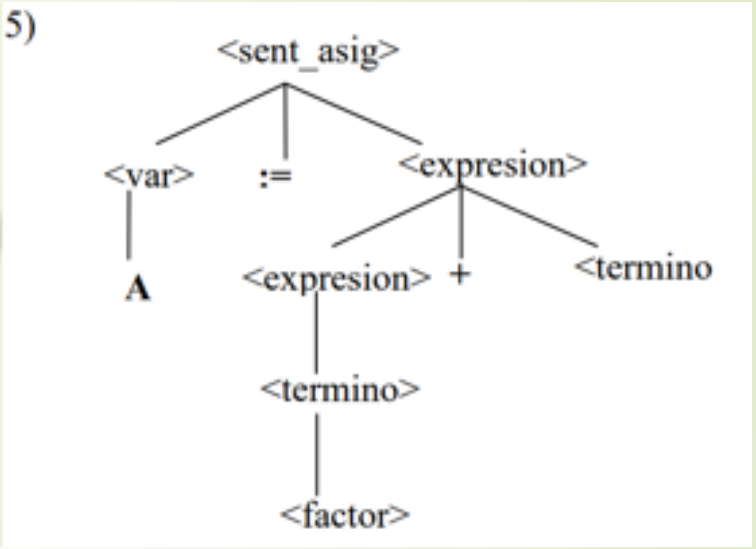
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

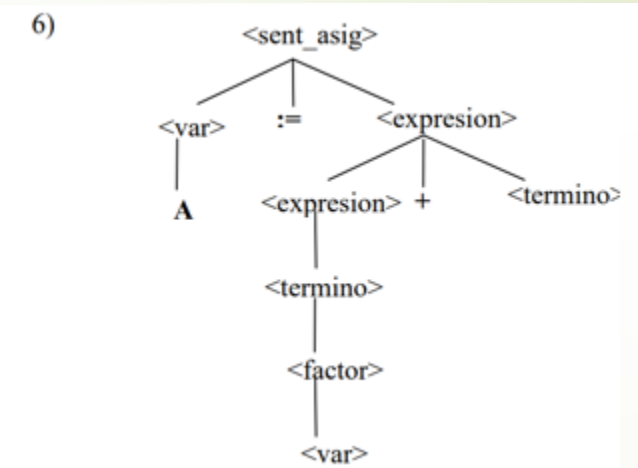
<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

5)



6)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

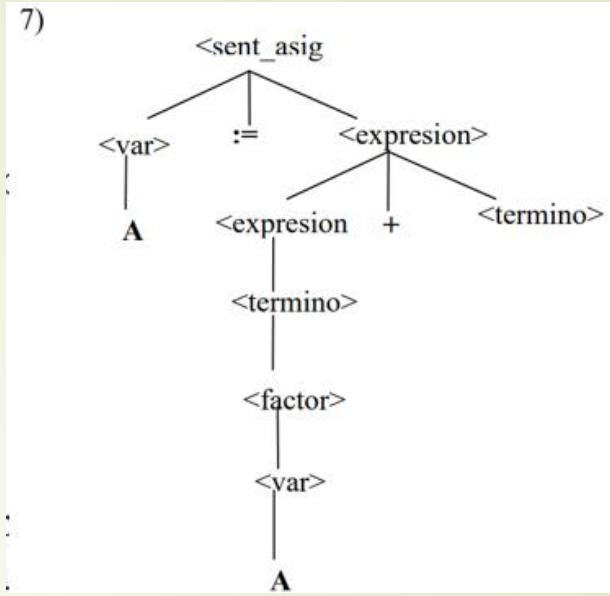
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

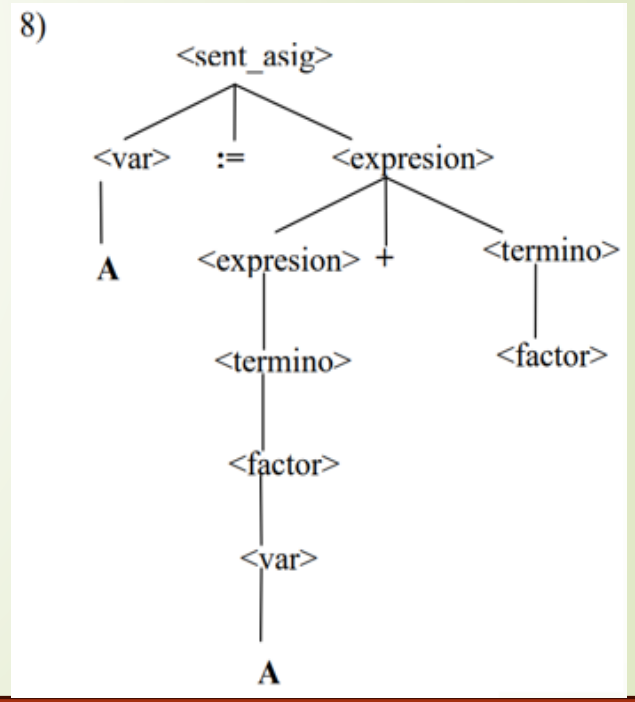
<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

7)



8)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

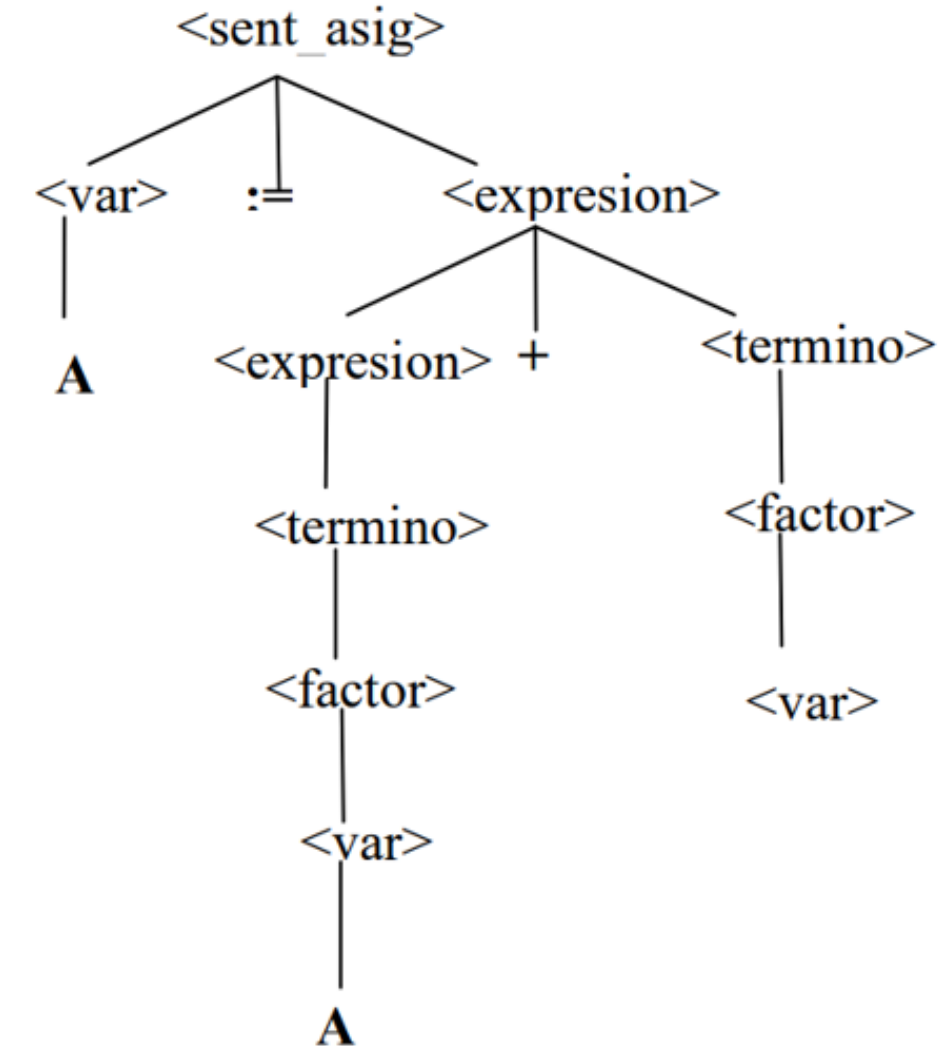
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

9)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Ejemplo 1 - Gramática

A := A+B

Ejemplo 6:

La siguiente definición BNF describe la sintaxis (simplificada) de una sentencia de asignación de un lenguaje tipo Pascal:

<sent_asig> ::= <var> := <expresion>

<expresion> ::= <expresion> + <termino> | <expresion> - <termino> | <termino>

<termino> ::= <termino> * <factor> | <termino> / <factor> | <factor>

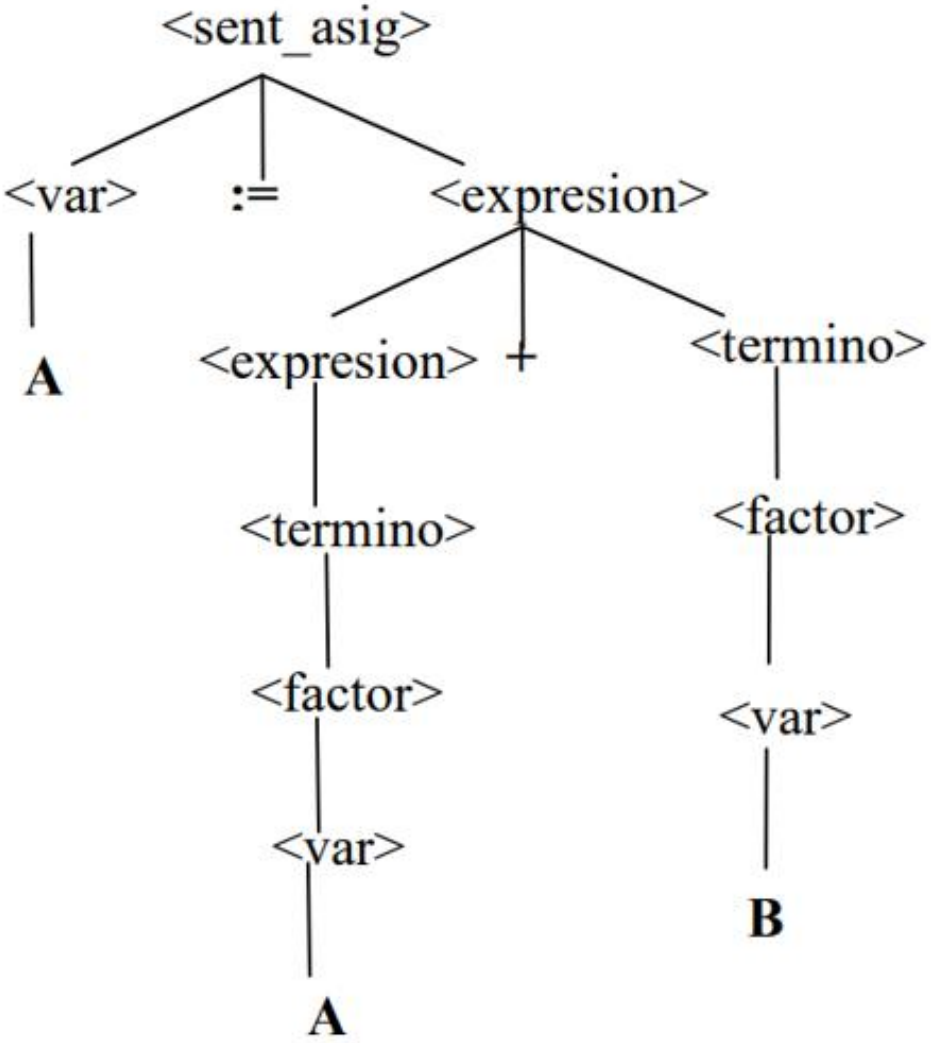
<factor> ::= (<expresion>) | <var> | <num>

<var> ::= A | B | C | D | ... | Z

<num> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Por ejemplo, la sentencia A := A + B es una sentencia de asignación que pertenece al lenguaje definido por la definición BNF dada, y cuyo árbol de derivación se construye como se muestra a continuación:

10)



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

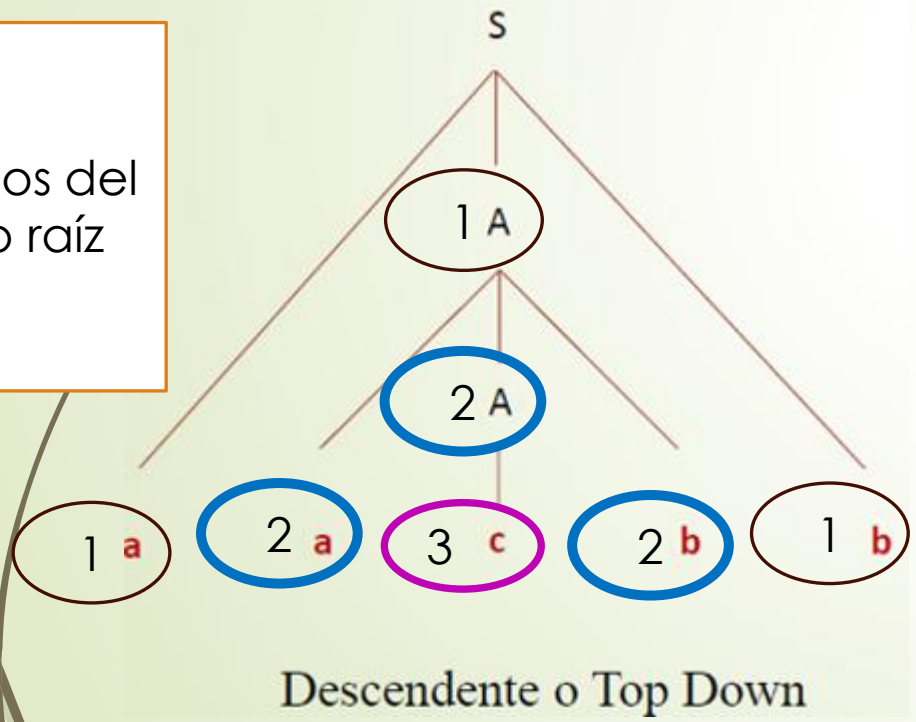
Ejemplo 2 - Gramática

1. S → a A b
2. A → a A b
3. A → c

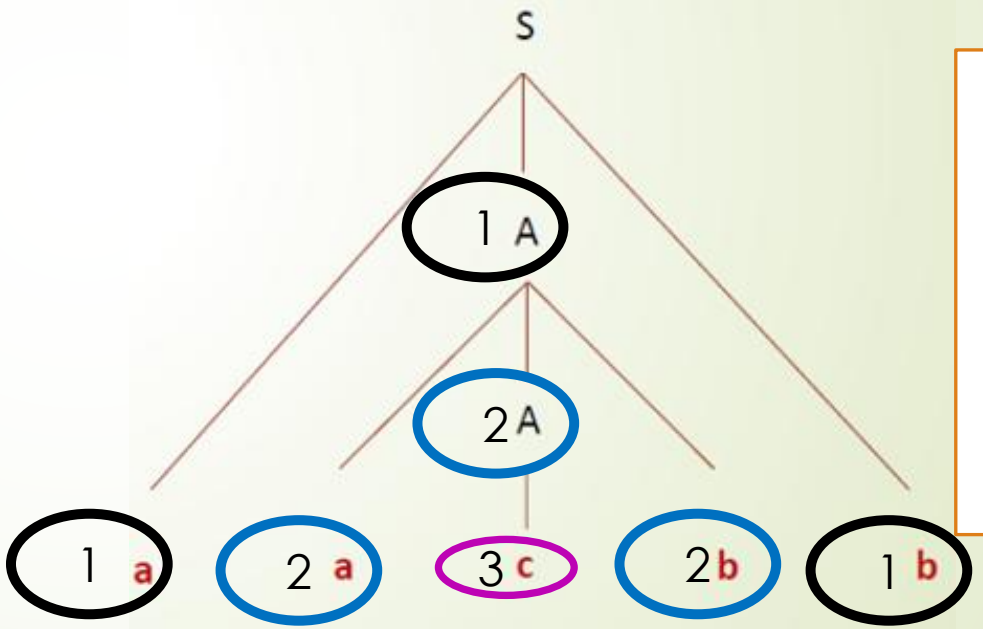
Hilera : **a a c b b**

Numeramos los pasos en los que se construye el árbol, los colores tienen relación con la regla aplicada

Partimos del nodo raíz



Descendente o Top Down

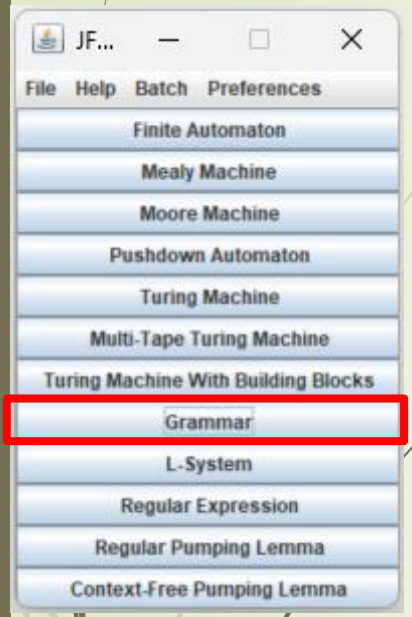


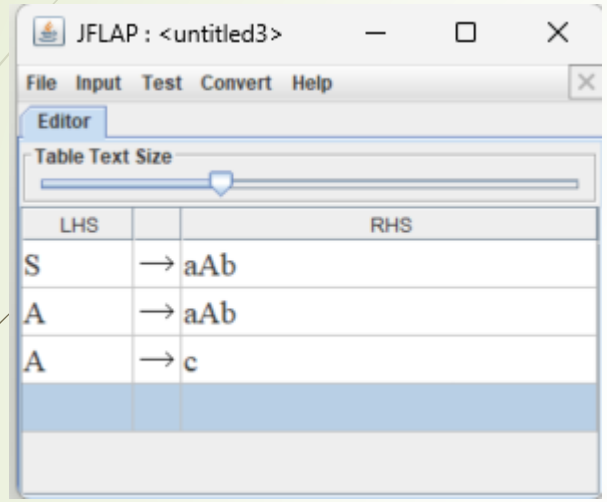
Ascendente o Bottom up

Partimos de los nodos hojas.
Elijo desde los E_N la regla más simple que pueda aplicar

Gramáticas en JFLAP

Lo vemos en JFLAP



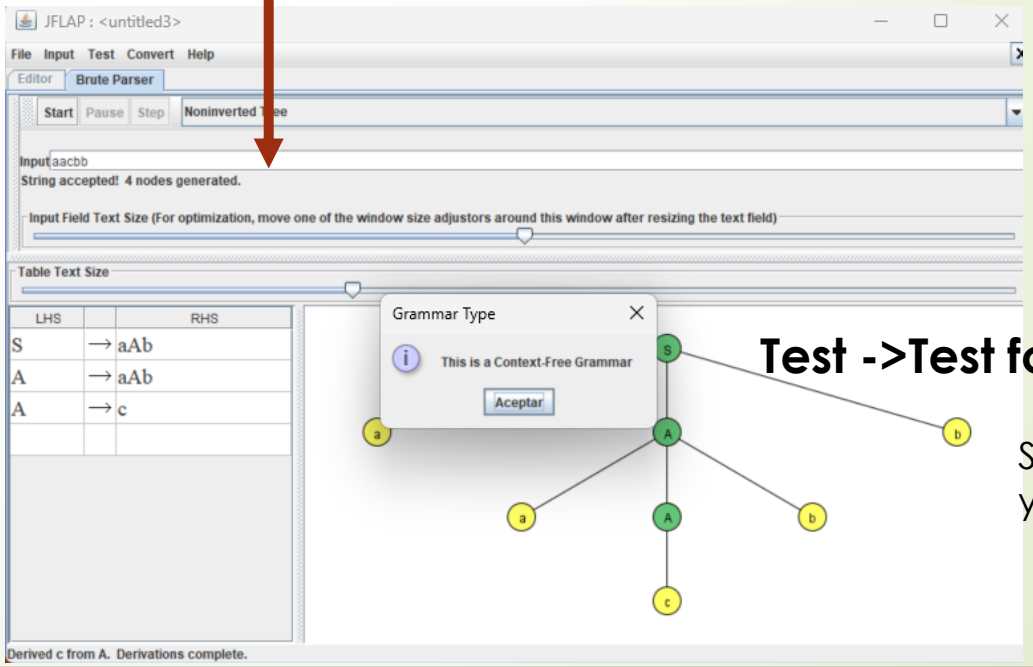


Hilera : a a c b b

1. S → a A b
2. A → a A b
3. A → c

Input->Brute Force Parser

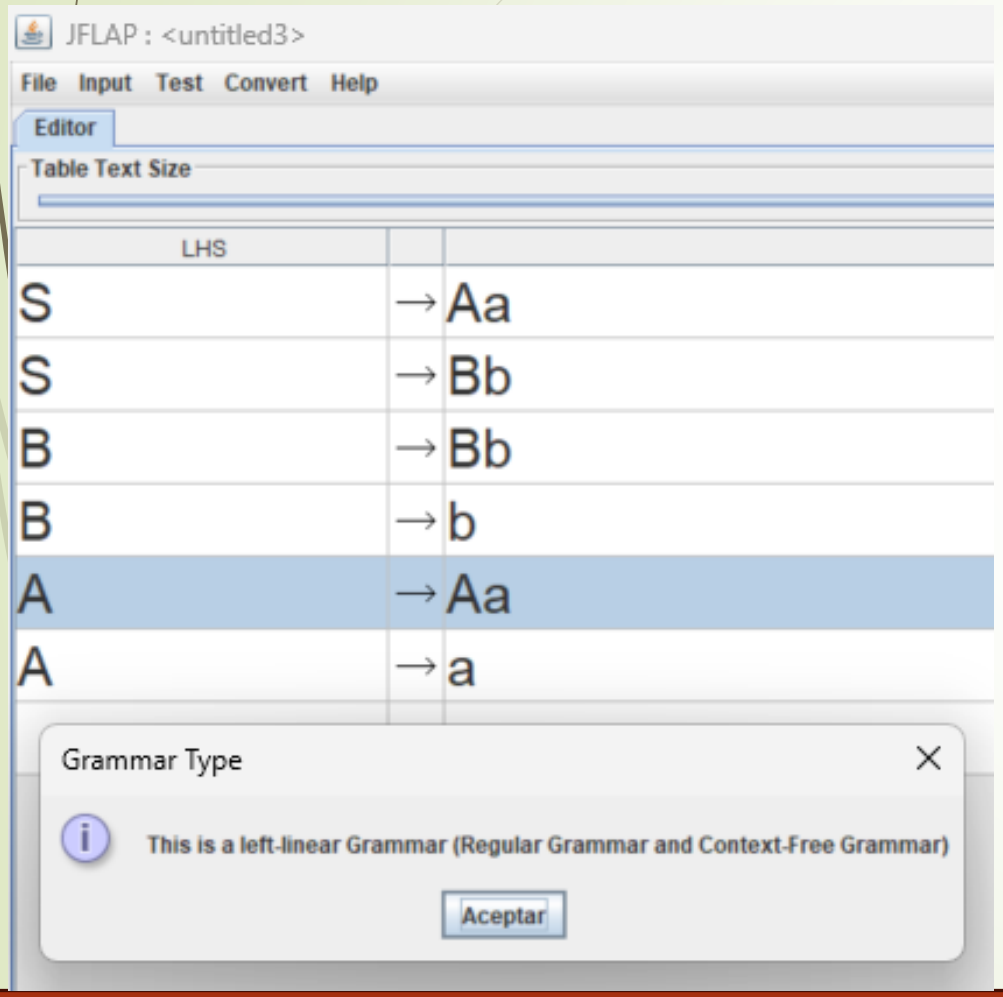
Se ingresa la cadena y nos arma el árbol

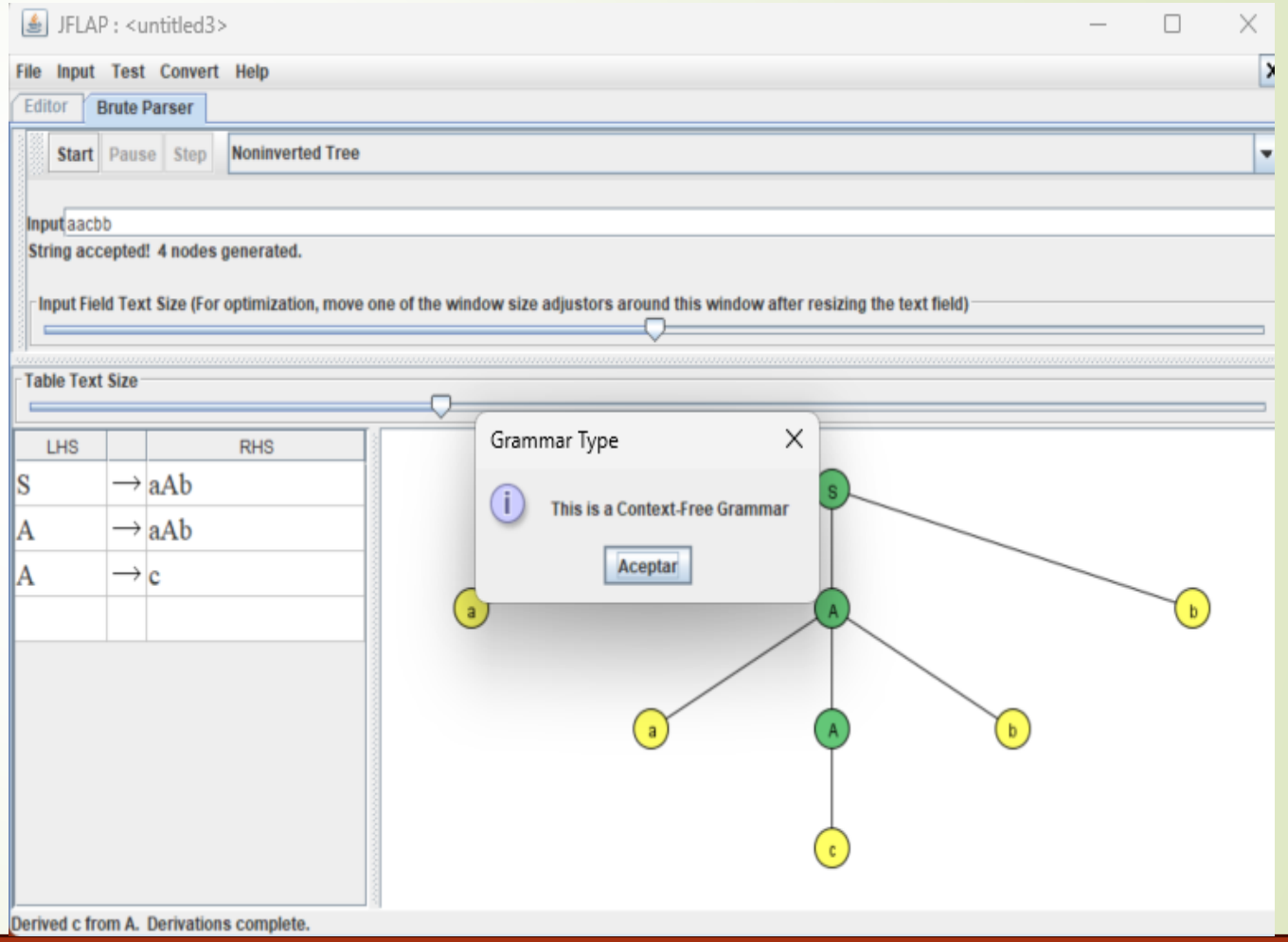


Test ->Test for Grammar Type

Se ingresa la cadena y nos arma el árbol

Gramáticas en JFLAP



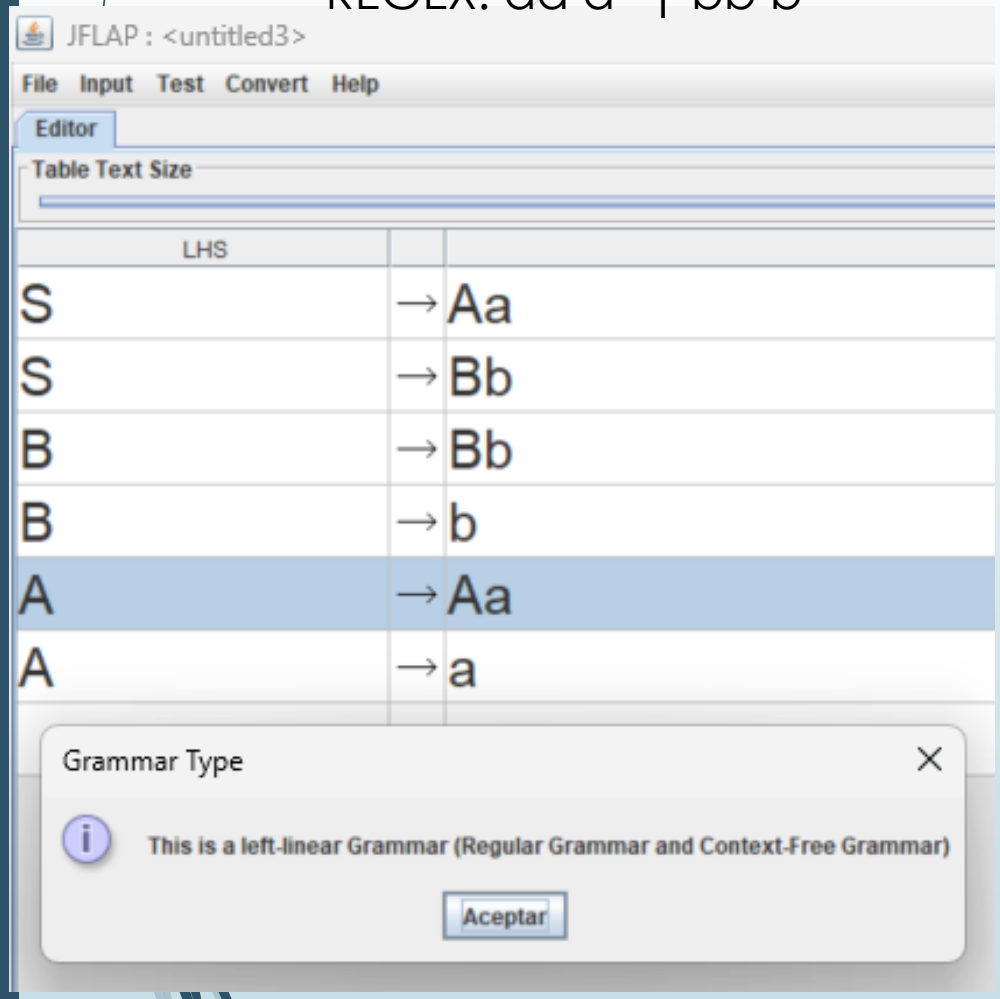


¿Cómo son los lenguajes?

Lenguaje Regular:

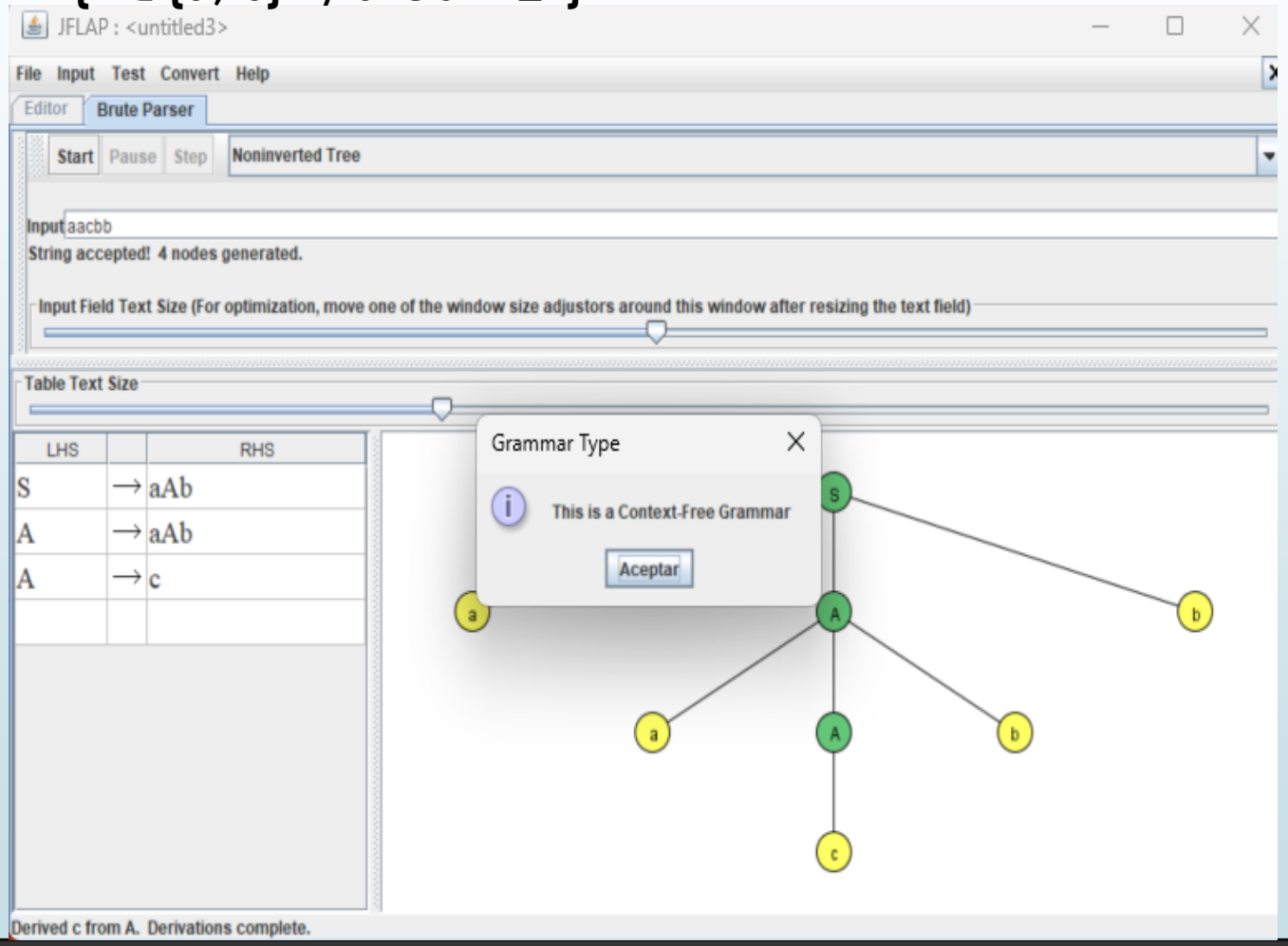
L={x ∈ {a, b}* / a^n, n≥2 ∪ b^m|m≥2}

REGEX: aa a* | bb b*

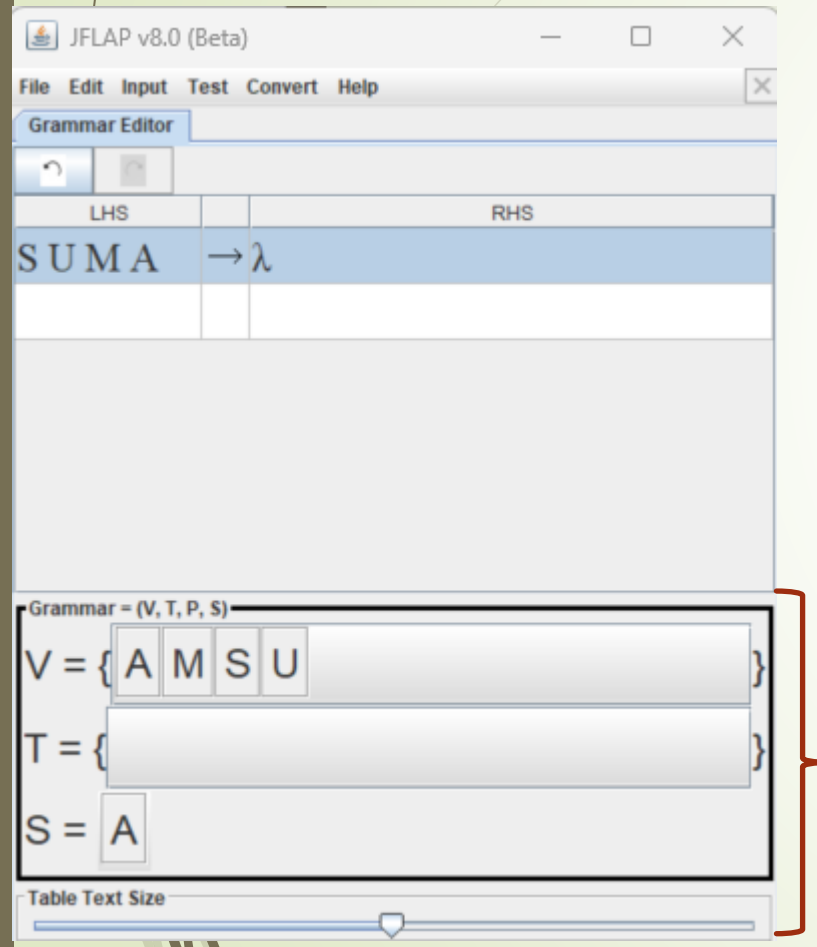


Lenguaje Libre de Contexto:

L={x ∈ {a, b}* / $a^n cb^n$ n≥1}



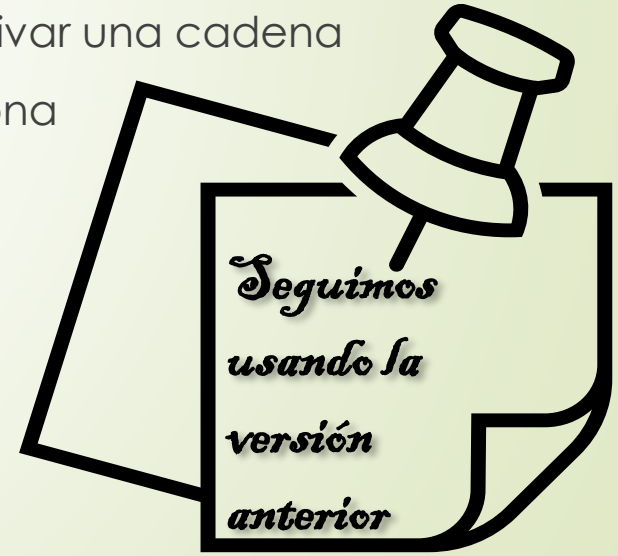
Gramáticas en JFLAP



- **No usa notación BNF**
  - Los terminales en minúsculas
  - Los no terminales en mayúsculas
  - No podemos usar palabras como SUMA… porque los tomará como un Conjunto Individual de NO TERMINALES
- Hay ocasiones en que nos exige que el símbolo distinguido esté en un nivel separado.
  - "$S \rightarrow E*V$" No deja probar si se puede derivar una cadena
  - "S → A A → E*V" De este modo sí funciona

En versión 8 muestra:

- V: las variables, NO TERMINALES
- T: Terminales
- S: Símbolo distinguido

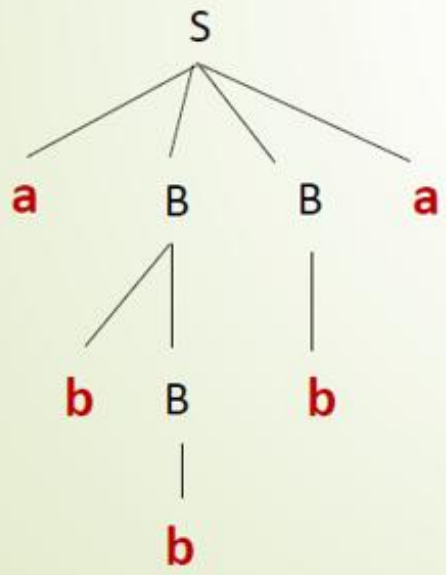


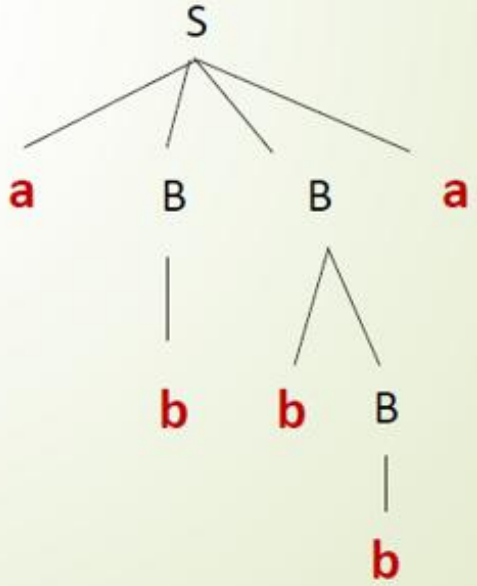
GIC – Ambigüedad (Ejemplo 1)

Una gramática es ambigua si existen más de un árbol de derivación para la misma hilera.

G= ({S,B}, {a,b}, P, S)

S → aBBa
B → b | bB





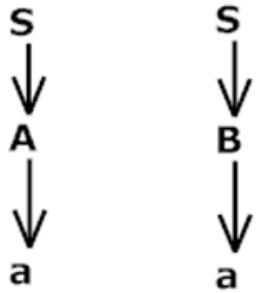
a b b b a

Gramáticas Ambiguas

La siguiente GIC es ambigua

S → A | B
A → aA | a
B → bB | a

Los árboles diferentes que derivan la misma palabra son:



"Las gramáticas se pueden clasificar en ambiguas y en no ambiguas. Una gramática es ambigua si y solo si existen dos o más árboles distintos que derivan la misma palabra. Las gramáticas ambiguas no son apropiadas para ser el modelo para la construcción de compiladores. Por ejemplo, no manejan bien la precedencia de los operadores".



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez
Sonia Jaramillo Valbuena
Sergio Augusto Cardona Torres
Armenia - Quindío

https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

Para cada cadena de un lenguaje generado por una gramática es posible construir (al menos) un árbol de derivación

Gramáticas

Ambiguas (una cadena tiene más de un árbol de derivación).

No ambiguas (cada cadena tiene un único árbol).

"Una gramática en la cual, para toda cadena generada w, todas las derivaciones de w tienen el mismo árbol de derivación es no ambigua. En algunos casos, dada una gramática ambigua, se puede encontrar otra gramática que produzca el mismo lenguaje pero que no sea ambigua. Si todas las gramáticas independientes del contexto para un lenguaje son ambiguas, se dice que el lenguaje es un lenguaje independiente del contexto **inherentemente ambiguo**".



UNICEN - Apunte de Cátedra – Ciencias de la Computación I
https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteGLC.pdf

Derivación Algoritmos

Los analizadores sintácticos (parsers) se basan en estas estrategias:

- Top-Down parsers (como los LL) usan derivación más a la izquierda. $S \Rightarrow aA \Rightarrow abB \Rightarrow abc$
- Bottom-Up parsers (como los LR) usan derivación más a la derecha. $abc \Rightarrow abB \Rightarrow aA \Rightarrow S$

Los parser leen la cadena desde la izquierda L:

- si derivan más a la izquierda LL
- si derivan más a la derecha LR

Estos algoritmos utilizan k símbolos de anticipación (lookahead) para decidir qué regla aplicar.

Por ejemplo, LL(1) mira el siguiente símbolo de la cadena para seleccionar la producción correcta, mientras que LR(k) puede mirar varios símbolos hacia adelante para resolver conflictos.

Algoritmos LL(k): construyen el árbol sintáctico siguiendo derivación más a la izquierda.

Algoritmos LR(k): reconstruyen la derivación más a la derecha en forma inversa.

CYK: no sigue estrictamente izquierda o derecha, sino que trabaja con subcadenas y gramáticas en FNC.

El algoritmo CYK lo veremos más adelante y también la FNC



Interpretar Gramáticas

Ejercicio Propuesto 1 (tomado del libro) ¿Cuál es el lenguaje representado por la siguiente gramática?

$S \rightarrow AB$ **La gramática se genera primero con A concatenado con B**

$A \rightarrow aaA \mid b$

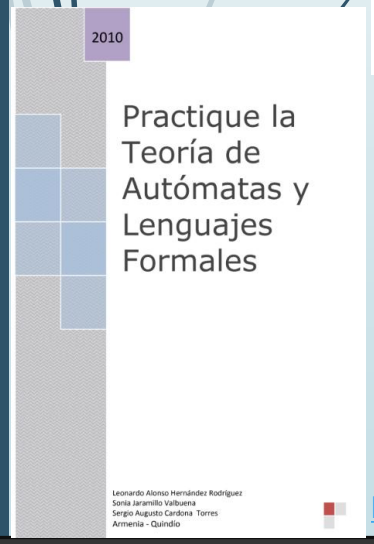
$B \rightarrow bBb \mid a$

Puede ser que A genere: $(aa)^n b$, con $n \geq 0$

- **sólo una b**
- **ó pares de a pero para cortar esos pares debe haber una b final**

Puede ser que B: $b^m a b^m$, con $m \geq 0$

- **genere sólo una a**
- **ó cadenas de igual cantidad de b delante que detrás con una a en medio para cortar la cadena**



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez
Sonia Jaramillo Valbuena
Sergio Augusto Cardona Torres
Armenia - Quindío

https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

a) $\{ (aa)^n bb^m ab^m \mid m \geq 0, n \geq 0 \}$

Ejercicio Propuesto 2 (tomado del libro) ¿Cuál es el lenguaje representado por la siguiente gramática?

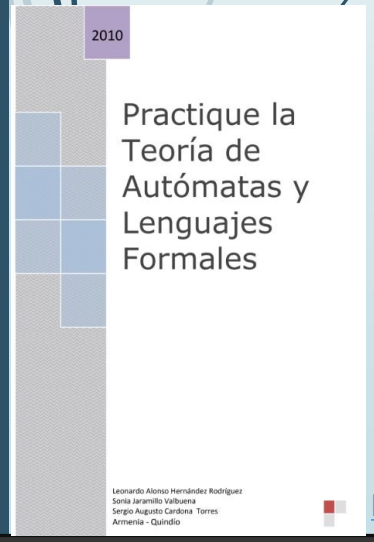
El lenguaje incluye la cadena vacía (se puede ver tanto en la opción A como B)

$S \rightarrow A \mid B \mid C$ **La gramática tiene tres alternativas**

$A \rightarrow aA \mid \lambda$ **Cadenas de a** $a^n \quad n \geq 0$

$B \rightarrow bbBa \mid \lambda$ **Cantidad pares de b y el doble de b que de a** $b^{2n} a^n \quad n \geq 0$

$C \rightarrow bbaC \mid bba$ **Grupos de bba todos los que se quiera** $(bba)^n \quad n \geq 1$



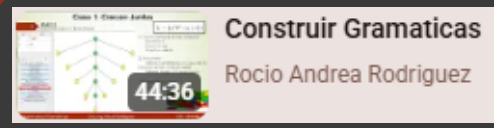
Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez
Sonia Jaramillo Valbuena
Sergio Augusto Cardona Torres
Armenia - Quindío

https://sara.uniquindio.edu.co/wp-content/uploads/2019/10/LHR-SJV-SACT-2010-Practique-Teor%C3%ADa-Autom%C3%A1tas-Lenguajes-Formales.pdf

$$L=\{a^n, n\geq 0\} \cup \{(b^{2n}a^n, n\geq 0\} \cup \{(bba)^n, n\geq 1\}$$

Construir Gramáticas

Casos Prácticos – Ejercicios



https://youtu.be/Wa0Kn-kuVFw?list=PLix2JrjhQgrHyIb27j7EtdXsqzp2DaqQK



Construir Gramáticas

Construir gramáticas - GIC



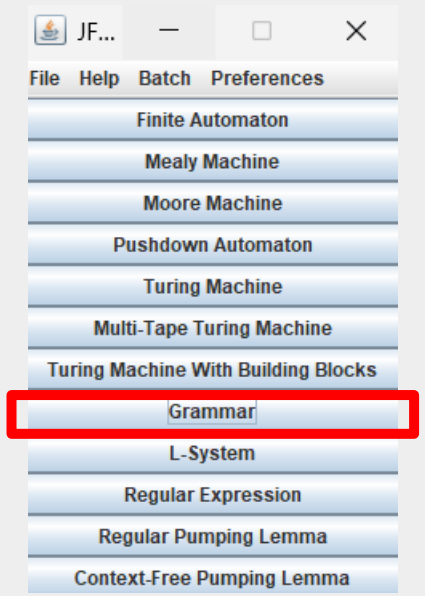
- Una **GIC no describe palabras**, describe **cómo se construyen**.
- Es importante pensar ¿cuáles son las cadenas que se pueden armar siguiendo la estructura del lenguaje?
- Dado un lenguaje L, lo importante es no empezar a escribir reglas, sin analizar primero:
  - Cadenas: ¿Qué cadenas pertenecen al lenguaje? Dar ejemplos de cadenas
  - Estructura: ¿Qué partes están relacionadas? ¿Qué partes crecen juntas? Los que crecen juntos se generan en la misma producción
  - Terminación: ¿Cuándo se termina? Caso base (cadena mínima, sin contar la cadena vacía) y Caso Recursivo (cómo se agranda)

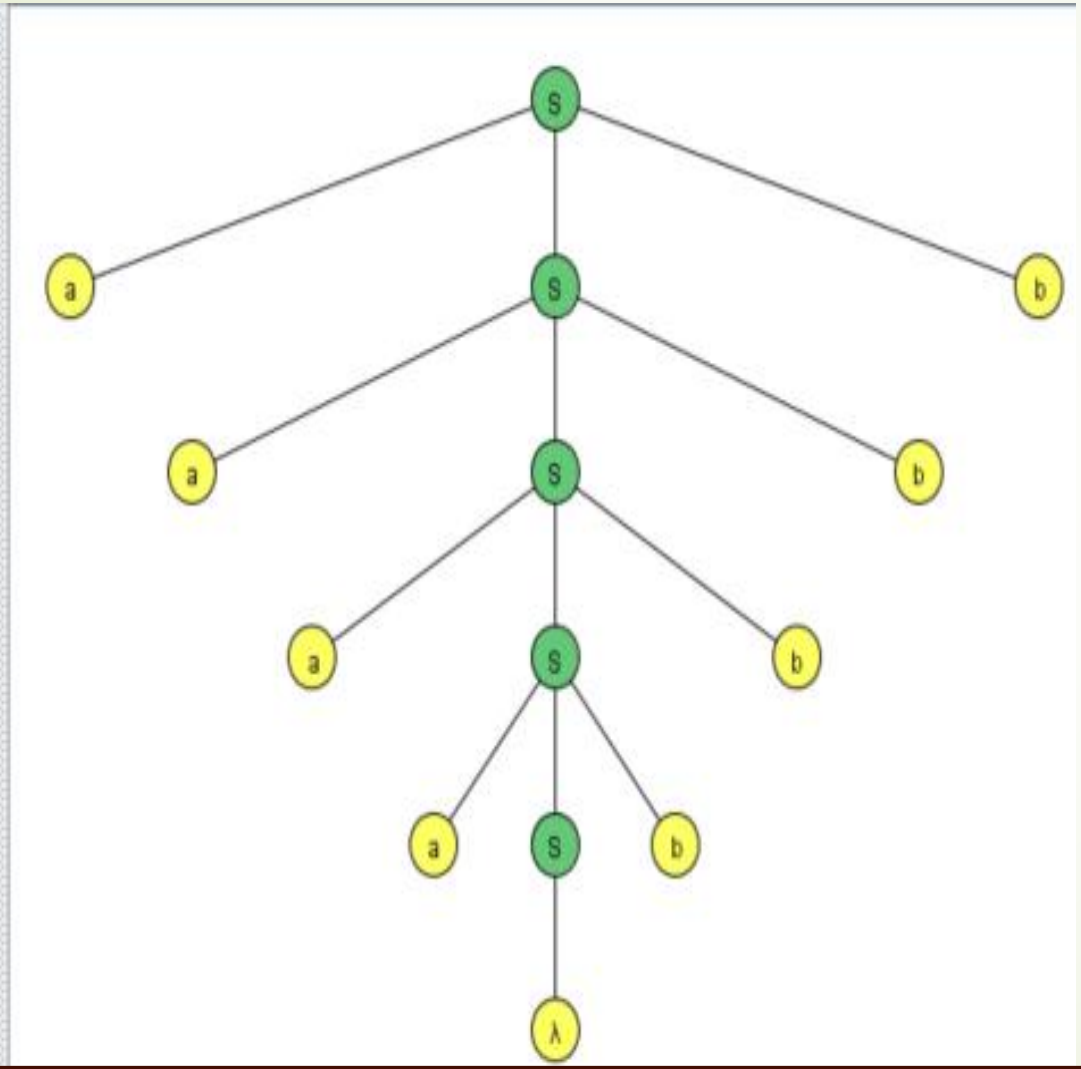
Caso 1: Crecen Juntos

jFLAP 7.1

Grammar: Input -> Brute Parser

LHS		RHS
S	→	aSb
S	→	λ





$$L = \{a^n b^n \mid n \geq 0\}$$

1) Forma general de las cadenas:
   Si n=0 => λ
   Si n=1 => ab
   Si n=2 => aabb

2) Estructura:
   Misma cantidad de a's que de b's (crecen juntas – misma regla)
   Primero todas a's luego las b's

3) Gramática (casos):
   S → a**S**b | λ



Caso 2: Crecen Separados

$$L = \{a^n b^m \mid n, m \geq 0\}$$

1) Forma general de las cadenas:
   Si n=0 m=0=> λ
   Si n=0 m=1 => b
   Si n=1 m=0 => a
   Si n=1 m=3 => abbb
   Si n=2 m=2 => aabb

2) Estructura:
   Los índices son independientes (no necesitan estar en la misma regla)

3) Gramática (casos):

S → **AB**

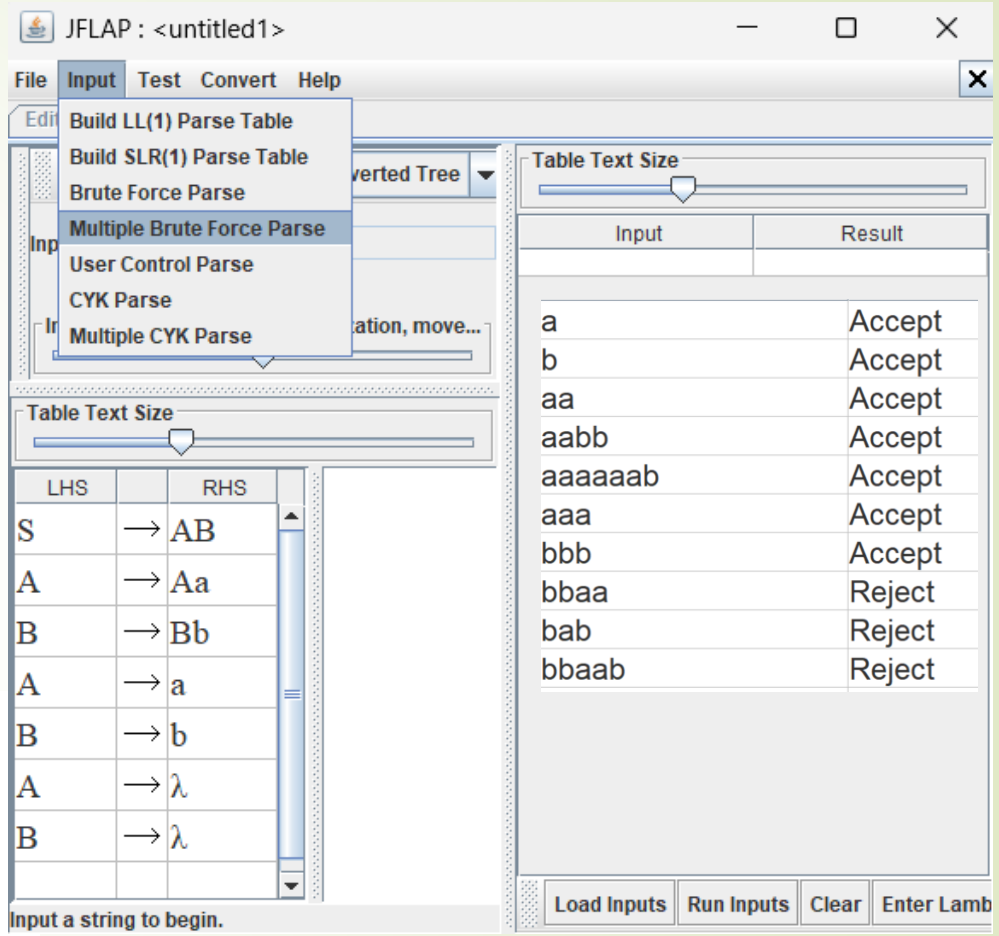
A → aA | a

B → bB | b

¿Cómo armo la cadena vacía?
¿Puede no tener a's y si tener b's?
¿Puede no tener b's y si tener a's?



¿Cuántos λ se necesitan?



S → **AB**

A → Aa | a | **λ**

B → Bb | b | **λ**

Caso 3: Índices Relacionados pero no iguales

$$L = \{ a^n b^{2n} \mid n \geq 0 \}$$

1) **Forma general de las cadenas:**

   λ
   abb
   aabbbb
   aaabbbbbb

2) **Estructura:**

   Por cada a hay dos b

3) **Gramática (casos):**

   S → aSbb | λ

Caso 4: Algo en el Medio

Ejemplo 3: $a^n c b^n$ **n ≥0**

1) **Forma general de las cadenas:**
   c
   acb
   aacbb
   aacbbb
   aaacbbb

2) **Estructura:**
   Las a's y las b's crecen juntas (van en la misma regla)
   Todas las cadenas terminan con una c en medio

3) **Gramática (casos):**
   S → aSb | c

Ejemplo 4: $a^n c b^n$ **n ≥1**

1) **Forma general de las cadenas:**
   acb (cadena mínima)
   aacbb
   aaacbbb
   aaaacbbbb
   aaaaacbbbbb

3) **Gramática (casos):**
   S → aTb
   T → aTb | c

Ejemplo 5: $a^n c b^n$ **n >1**

Ahora ustedes!!!!

S → **aa**T**bb**
T → aTb | c

Caso 5 – Lo del medio relacionado con los extremos

$$L = \{ a^n b^{n+m} c^m \mid n, m \geq 0 \}$$

1) Palabras pertenecientes al lenguaje:

λ
abbc
bc (n=0 y m=1)
ab (n=1 y m=0)

2) Estructura:

La cantidad de a y c pueden ser distintas pero la cantidad de b tiene que ser la misma que la cantidad de a's+c's

Entonces por cada a tengo que tener una b y por cada c tengo que tener una b

3) Gramática:

S → AB
A → **a**A**b** | λ
B → **b**B**c** | λ

Ejemplo de Construcción de GIC Video de Tarea

- Universidad de La Laguna – España

  Prof. Gara Miranda Valladores

  https://youtu.be/w6hDnPYpzdc



Usa ε en vez de λ

$$L = \{ a^m b^n c^p d^q \mid m + n \geq p + q \}$$

Algunos ejemplos más, los intentamos hacer menos detallados

Ejemplo 1 – Binario Palíndromo

$S \rightarrow 0\mathbf{S}0 \mid 1\mathbf{S}1 \mid 0 \mid 1$ → Permite de un bit
No permitiría cantidad pares de dígitos, por ejemplo longitud 2 ó longitud 4

$S \rightarrow 0\mathbf{S}0 \mid 1\mathbf{S}1 \mid 0 \mid 1 \mid \lambda$ → Permite de un bit
Permite pares, pero también admite λ

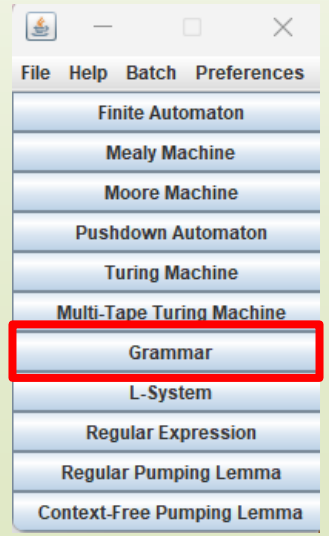
Ejemplos:
0110
1001
010
101
11
00

¿Sino considero a λ como parte del lenguaje como puedo quitarla?

$S \rightarrow 0\mathbf{A}0 \mid 1\mathbf{A}1 \mid 0 \mid 1$
$A \rightarrow 0\ A\ 0 \mid 1\ A\ 1 \mid 0 \mid 1 \mid \boldsymbol{\lambda}$

¿Y si quisiera considerar palíndromos a partir de longitud 2?

$S \rightarrow 0\mathbf{A}0 \mid 1\mathbf{A}1$
$A \rightarrow 0\ A\ 0 \mid 1\ A\ 1 \mid 0 \mid 1 \mid \boldsymbol{\lambda}$



Ejemplo 2

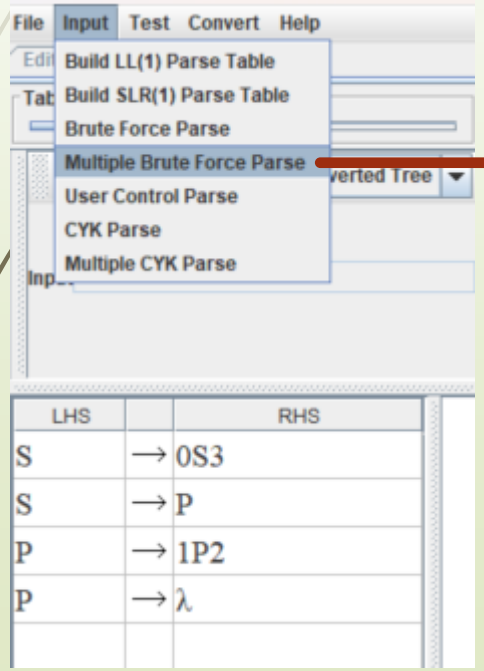
$\{0^n\ 1^m\ 2^m\ 3^n$ con n,m≥0 $\}$

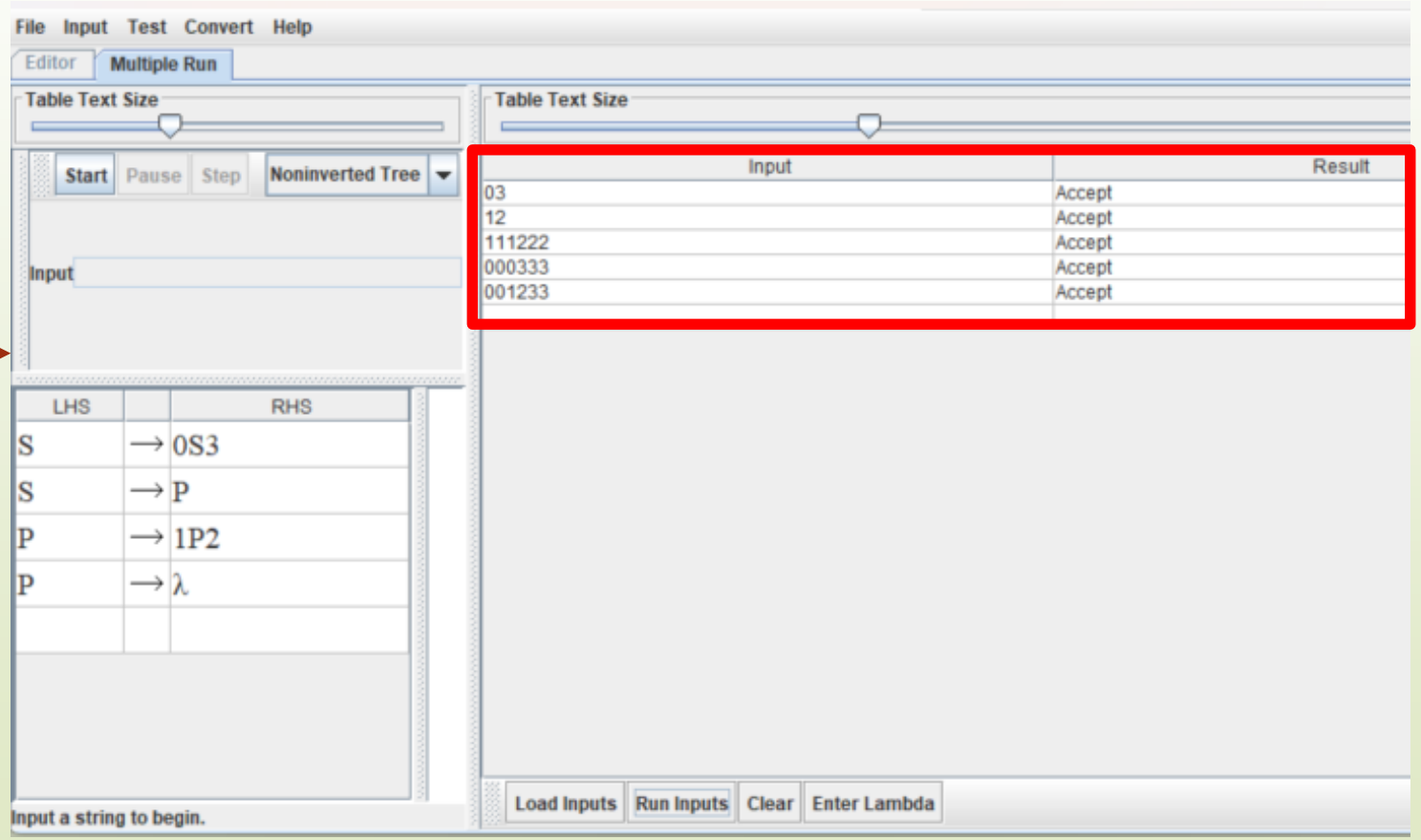
P

S

- S (extremos balancea los 0's y los 3's)
- P (parte del medio, que puede estar sin tener los extremos es decir n=0)

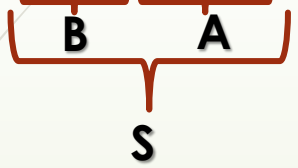
S → 0S3 | P
P → 1P2 | λ





Ejemplo 3

$\{0^n\ 1^n\ 2^m\ 3^m$ con n,m≥0 $\}$



S → BA | λ → **¿Es necesario?**
B → 0B1 | λ
A →2A3 | λ

Resultado
S → BA
B → 0B1 | λ
A →2A3 | λ

Ejemplo 4

$\{0^m\ 1^n$ con m ≥ n$\}$

Para la parte del igual misma cantidad de 0's que de 1's ¿como haríamos?

S → 0**S**1

Ahora hay que ver como lograr que haya más 0 que 1

S → 0**S**

Resultado
S → $0S1$ | **0S** | λ

Y si fuera mayor estricto:

$\{0^m\ 1^n$ con m > n$\}$

$$S \rightarrow 0S1 \mid 0S \mid 0$$

Ejercicio escribir la gramática

$\{0^m\ 1^n$ con n,m≥0 $\}$

C U

S

00
111
00001
00111
λ

LHS		RHS
S	→	CU
C	→	0
C	→	0C
U	→	1
U	→	1U
C	→	λ
U	→	λ

Table Text Size

Input	Result
00000	Accept
11111	Accept
0	Accept
1	Accept
001111	Accept
0000001	Accept

Gramáticas en Formas Normales

- Forma Normal de Chomsky **(FNC)**
- Forma Normal de Greibach **(FNG)**

Forma	¿Para qué sirve?
FNC	Algoritmo CYK
FNG	Parsers (análisis descendente - consumo garantizado de terminales en cada derivación)

Toda gramática libre de contexto puede transformarse en una gramática equivalente en FNC ó FNG, preservando el lenguaje original.



Noam Chomsky

Nacido en Estados Unidos en 1928.

- Licenciado en Lingüística
- Maestría en Lingüística
- Doctorado en Lingüística

Es considerado uno de los padres de la lingüística moderna.

Sheila Adele Greibach

Nacida en Estados Unidos en 1939

- Licenciada en Lingüística y Matemáticas
- Maestría en Matemáticas Aplicadas
- Doctorado en Matemáticas Apicadas



Características de FNC

Estructura de las producciones

- Cada regla de producción tiene una de estas formas:
  - $A \rightarrow BC$ (dos símbolos no terminales)
  - $A \rightarrow a$ (un símbolo terminal)
- Opcionalmente, la producción $S \rightarrow \lambda$ (solo si la gramática genera la cadena vacía), donde **S** es el símbolo inicial.

Restricciones

- No se permiten producciones con más de dos no terminales, ni mezclas de terminales y no terminales en la misma regla.
- No se permiten producciones vacías (excepto la mencionada para el símbolo inicial) ni producciones unitarias (del tipo $A \rightarrow B$).

Características de FNG

Estructura de las producciones

- Cada regla de producción tiene la forma: $A \rightarrow a\,\boldsymbol{N}$ donde:
  - A es un símbolo no terminal.
  - a es un símbolo terminal (la regla siempre comienza con un terminal).
  - **N** es una secuencia (posiblemente vacía) de símbolos no terminales.

Restricciones

- No se permiten producciones que comiencen con un NO TERMINAL.
- No se permiten producciones vacías (excepto en casos especiales para el símbolo inicial).
- No se permiten producciones unitarias (del tipo A → B).

Ejemplo

Dada la siguiente gramática reescribirla en FNG y FNC

S → A b
A → a | a A

En **FNG**, cada producción debe comenzar con un **terminal**, seguido de cero o más no terminales.

S → A b
A → a | a A

1. Regla: S → A b

Si sustituimos A por sus producciones:
S → a b | a A b

Tienen un terminal delante pero aún no cumple
Entonces usamos un No Terminal adicional: **Y**

2. Regla: A → a | a A
Ya cumple la condición:
A → a | a A

S → a **Y** | a A **Y**
A → a | a A
Y → b

Ejemplo

Dada la siguiente gramática reescribirla en FNG y FNC

S → A b
A → a | a A

En **FNC**, cada producción debe tener:
A → BC (dos no terminales),
A → a (un terminal),
con la excepción opcional de S → λ si la gramática genera la cadena vacía.

1. Introducir EN para terminales
Creamos nuevos no terminales para cada terminal:

X→ a
Y→ b

2. Reescribir las reglas que no están en FNC, reemplazando por los nuevos EN

S → A**Y**
A → a | **X**A

S → A Y
A → a | XA
X→ a
Y→ b

Ejemplos de Normalización en FNC

Ejercicio 1 - Normalización – FNC (enunciado y resolución tomado de un libro)

Elabore una GIC en forma normal de Chomsky para el lenguaje $\{a^{2i+1} | i \geq 0\}$

¡RECORDATORIO!

Estructura de las producciones: Cada regla de producción tiene una de estas formas:

1. $A \rightarrow BC$ (dos símbolos no terminales)
2. $A \rightarrow a$ (un símbolo terminal)

Opcionalmente, la producción $S \rightarrow \lambda$ (solo si la gramática genera la cadena vacía), donde **S** es el símbolo inicial.

$$S \rightarrow aSa|a$$

Reemplacemos los terminales de las producciones que no están en forma normal de Chomsky, por no terminales. En este caso, reemplazamos las aes de la primera producción, por A. La gramática queda:

$$S \rightarrow ASA|a$$

$$A \rightarrow a$$

En las producciones que tengan más de 2 no terminales, reemplazamos grupos de 2 no terminales por un no terminal nuevo. En este caso, reemplazamos por ejemplo AS por B y la gramática ya queda en forma normal de Chomsky:

$$S \rightarrow BA|a$$

$$B \rightarrow AS$$

$$A \rightarrow a$$

Ejercicio 2 - Normalización – FNC (enunciado tomado de un libro)

Elabore GIC en forma normal de Chomsky para el lenguaje:

$$\{(ab)^i a^i \mid i \geq 1\}$$

S→ abSa | aba

¡RECORDATORIO!

Estructura de las producciones: Cada regla de producción tiene una de estas formas:

1. $A \rightarrow BC$ (dos símbolos no terminales)
2. $A \rightarrow a$ (un símbolo terminal)

Opcionalmente, la producción $S \rightarrow \lambda$ (solo si la gramática genera la cadena vacía), donde **S** es el símbolo inicial.

(1)Reemplacemos los terminales de las producciones que no están en FNC por E_N

A → a

B → b

S → **ABSA** | **ABA**

A → a

B → b

(2)Las que tienen más de dos E_N se vuelven a reemplazar por un E_N intermedios (para dejarlos en grupos de a dos)

~~S →~~ ~~**ABSA**~~

S → **AX**

X → B**Y**

Y→ **SA**

A → a

B → b

~~S →~~ ~~**ABA**~~

S → A**Z**

Z → **BA**

Resultado

S → AX | AZ

X → BY

Y → SA

Z → BA

A → a

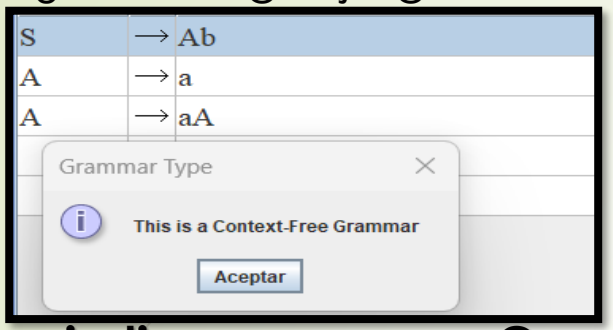
B → b



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez, Sonia Jaramillo Valbuena, Sergio Augusto Cardona Torres
Armenia – Quindío, Colombia

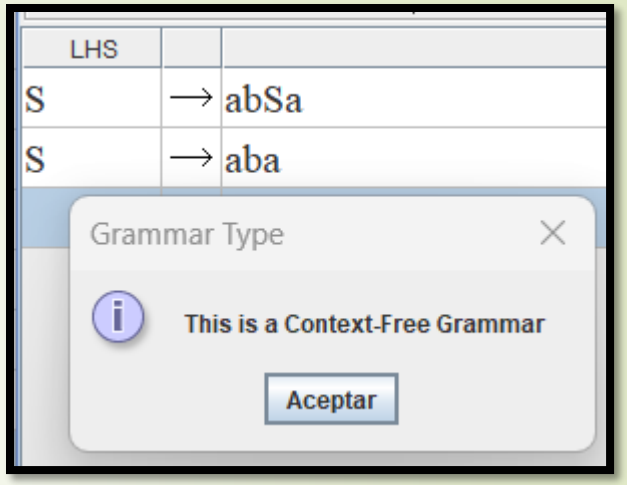
FN en JFLAP

¿Qué lenguaje genera?

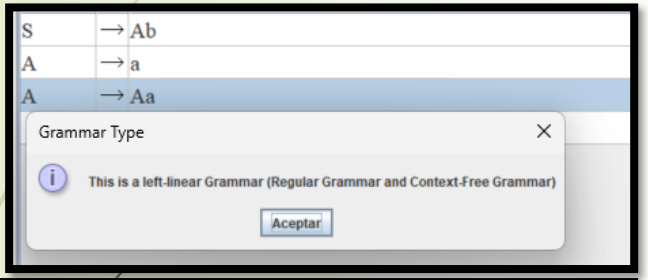


a*b

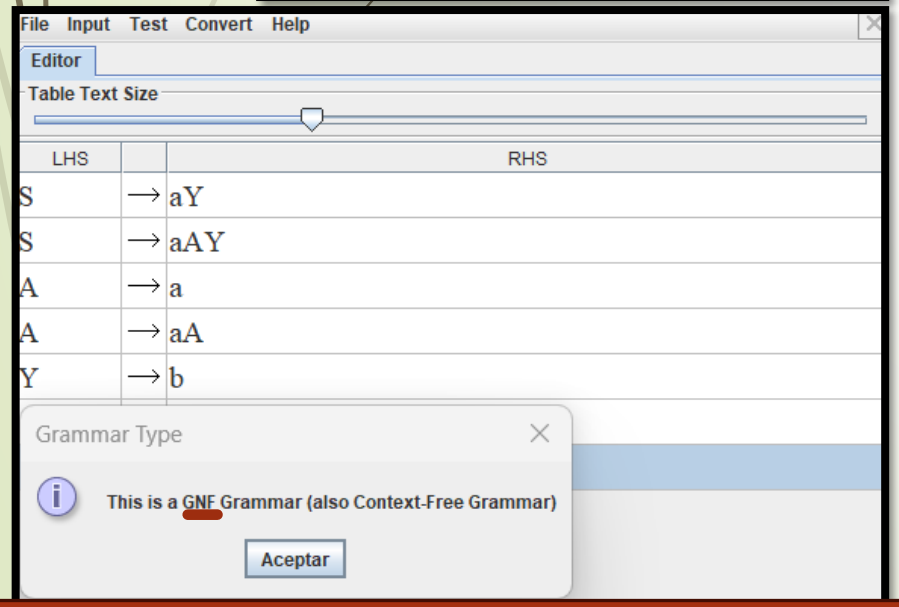
Probamos la gramática inicial
Y luego la gramática normalizada

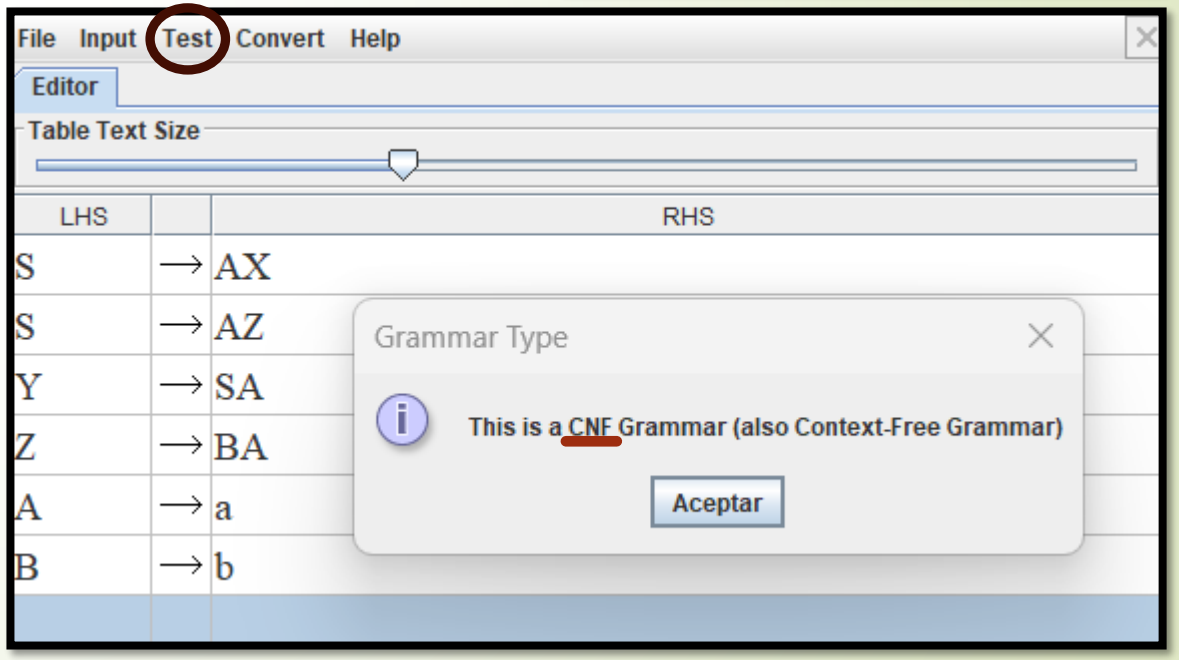


¿Porqué no indica que es una Gramática Regular?



¡Elegir recursividad a derecha ó a izquierda, sin mezclar!





Algoritmo CYK

Permite validar si una cadena pertenece o no a una gramática



https://youtu.be/q_hA5htrlqU

Algoritmo CYK
Árbol de Derivación

- **Árbol de derivación**:

Se puede construir con cualquier GIC, incluso si no está en Forma Normal de Chomsky (FNC).

Basta con aplicar las reglas de la gramática para generar la cadena y mostrar la estructura jerárquica.

- **Matriz CYK**:

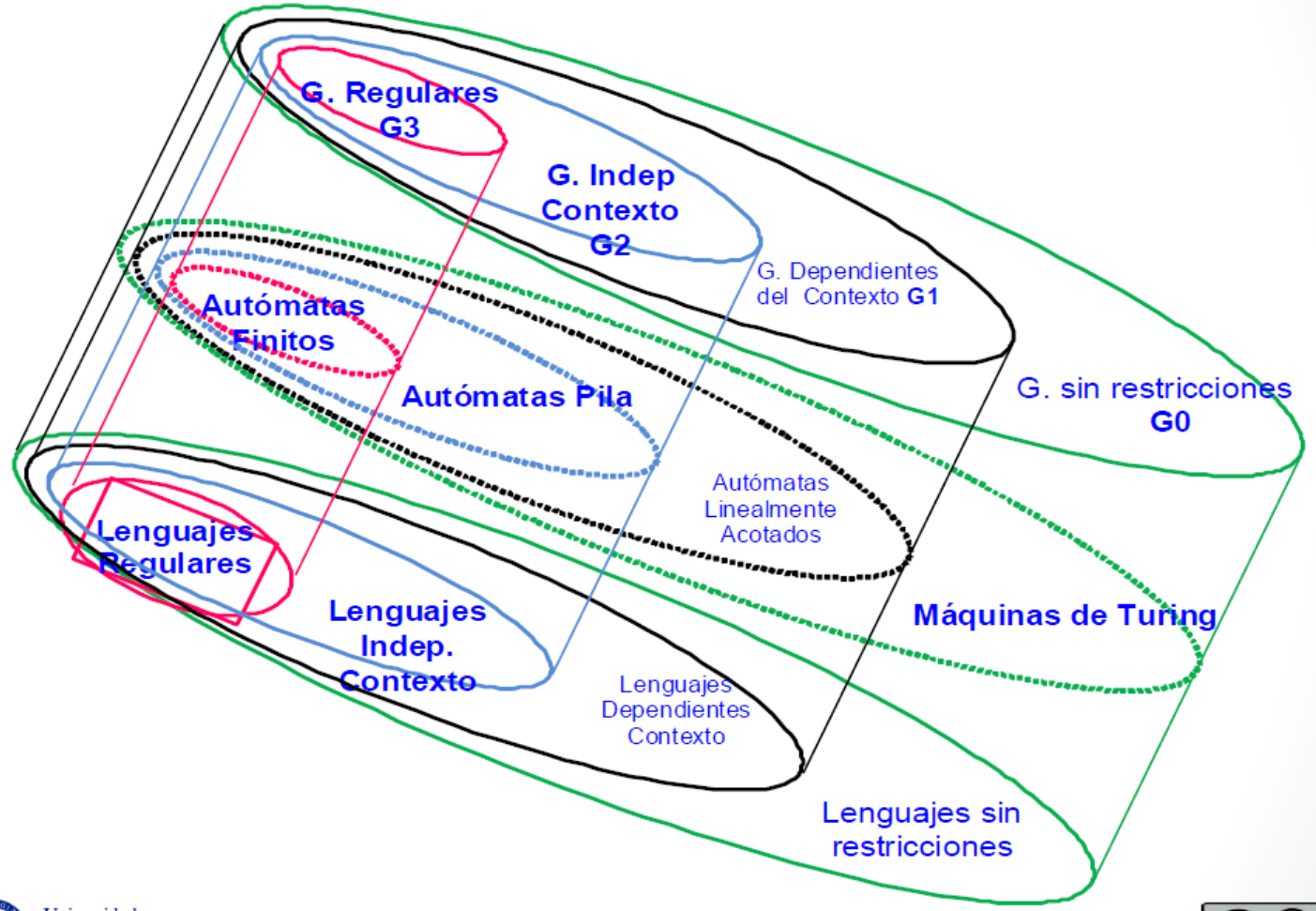
El algoritmo CYK requiere obligatoriamente que la gramática esté en FNC porque su lógica se basa en reglas del tipo:

- $A \rightarrow BC$ (dos no terminales)
- $A \rightarrow a$ (un terminal)

Si la gramática no cumple esto, no se puede aplicar el algoritmo directamente.

Los LIC son descriptos mediante una GIC, pero no pueden reconocer las cadenas del lenguaje por medio de un AF (AFD o AFN) sino que requiere un Autómata con mayor complejidad, un Autómata con Pila (AP)

Máquinas, Lenguajes y Algoritmos



Universidad
Carlos III de Madrid
www.uc3m.es

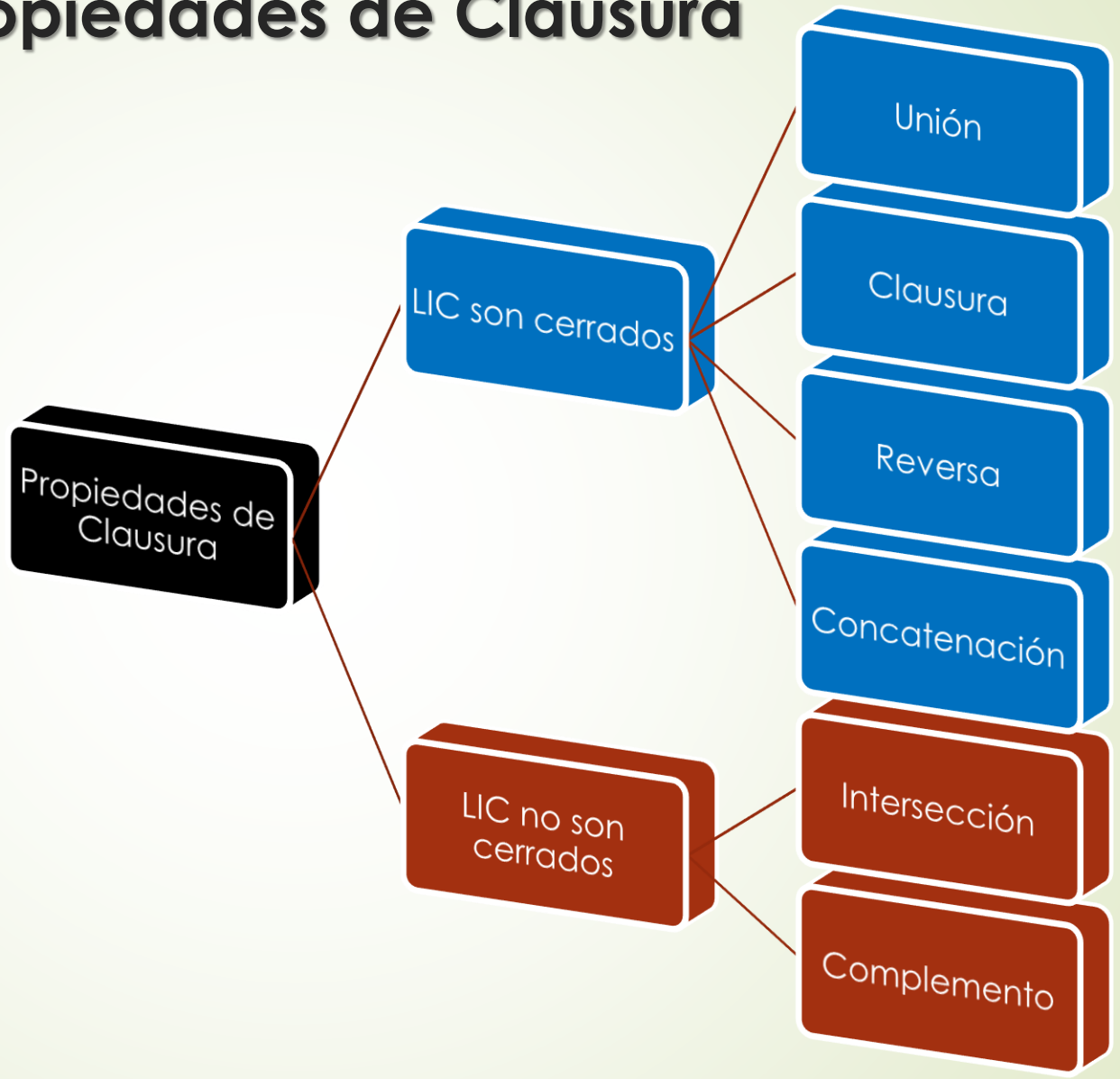
cc BY NC SA

A. Sanchis, A. Ledezma, J.A. Iglesias, B. García, J. M.Alonso

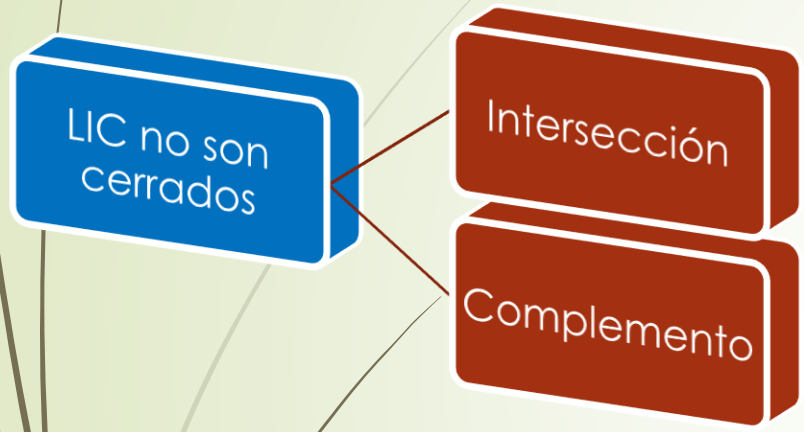
Lenguajes Independientes de Contexto Propiedades de Clausura

- En teoría de lenguajes formales, las propiedades de clausura se llaman así porque indican que un conjunto (en este caso, el conjunto de lenguajes independientes de contexto) es *cerrado* respecto a ciertas operaciones. Esto significa que si tomamos uno o más lenguajes del mismo tipo y aplicamos una operación específica, el resultado sigue siendo un lenguaje del mismo tipo.

Lenguajes Independientes de Contexto
Propiedades de Clausura



Lenguajes Independientes de Contexto
Revisión de las Propiedades de Clausura



Los Lenguajes Independiente de Contexto (LIC) que ya no pueden describirse con una REGEX (requieren de índices).
Sus cadenas que ya no pueden validarse con un AFD o AFND porque requieren contar cantidades. Entonces se requiere un Autómata con Pila (AP) en donde para L1 pueda apilar las a's que leo y luego por cada b desapilar una a, con la última b leída la pila debe quedar vacía.

$L_1 = \{ a^n b^n c^m \quad n, m \geq 0 \}$

$L_2 = \{ a^m b^n c^n \quad m, n \geq 0 \}$

L1 es un LIC
L2 es un LIC

$\mathbf{L_1 \cap L_2 = \{ a^n b^n c^n \quad n \geq 0 \}}$

L1 ∩ L2 **no** es un LIC

Con una pila **no** puedo validar iguales cantidades de a que de b que de c

El complemento de un lenguaje implicaría realizar un autómata que detecte todas las cadenas que no pertenecen al lenguaje: Detectar todo lo que no se cumple en L3 requiere mayor poder, no se puede con una pila.

L3 es un LIC

- Apillo a's
- Apilo b's
- Con cada c desapilo una b
- Con cada d desapilo una a

$\mathbf{L_3=\{a^n b^m c^m d^n \quad n, m \geq 0\}}$

Dentro del complemento de L3 podría estar este tipo de cadenas definidas por un lenguaje L

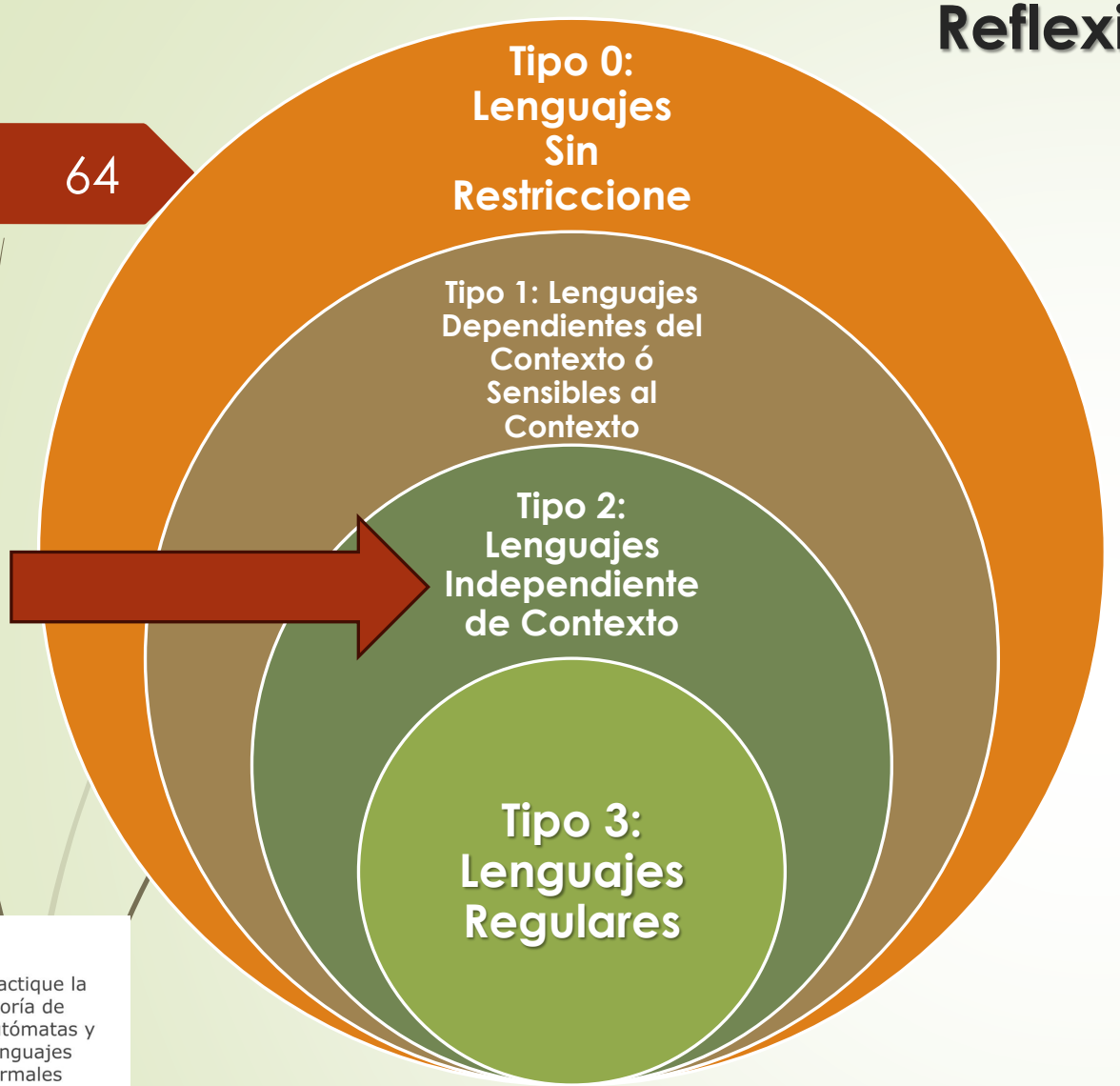
$L=\{a^n b^m c^n d^m \quad n, m \geq 0\}$

L **no** es un LIC

- Puedo apilar a's
- Puedo apilar b's

Cuando leo c ya no puedo desapilar a porque lo que tengo en el tope de la pila son b
No lo puedo resolver con una pila

Reflexiones

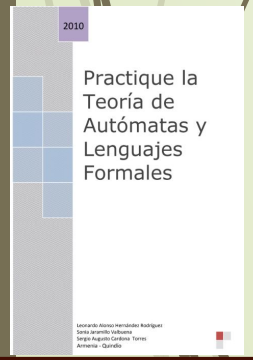


"Los lenguajes independientes del contexto, LIC, son un conjunto más grande que los lenguajes regulares y los incluyen. En realidad, los lenguajes regulares son solo una pequeña parte de los lenguajes independientes del contexto. Aun así, los lenguajes independientes del contexto no son todos los lenguajes posibles, aún quedan lenguajes por fuera de este conjunto".

Un ejemplo de un lenguaje regular y por lo tanto independiente del contexto es $\{a^i \mid i \geq 0\}$.

Un ejemplo de un lenguaje independiente del contexto que no es regular es $\{a^i b^i \mid i \geq 0\}$.

Un ejemplo de un lenguaje que es no es ni regular, ni independiente del contexto, es $\{a^i b^i c^i \mid i \geq 0\}$



Libro disponible digital: Practique la Teoría de Autómatas y Lenguajes Formales
Leonardo Alonso Hernández Rodríguez, Sonia Jaramillo Valbuena, Sergio Augusto Cardona Torres
Armenia – Quindío, Colombia

Fin

GIC